

פרק 11: אלגוריתם K-Means – למידה סטטיסטית וכריית מידע

Chapter 11: K-Means Algorithm – Statistical Learning and Data Mining

ד"ר יורם סגל

© Dr. Segal Yoram - כל הזכויות שמורות

אוקטובר 2025

גרסה 2.0

תודה מיוחדת לפרופ' דני וילנציק על ההרצאות המעוררות השראה
שעליהן מבוסס פרק זה.

תוכן העניינים

7	1 *
8	2 בעיית הקלסטרינג: מסע אל עולם הלמידה הלא-מפוקחת
8	2.1 פילוסופיה של קטגוריזציה
8	2.2 הגדרה מתמטית פורמלית
9	2.3 מטריקות מרחק במרחבים מטריים
9	2.4 דוגמה קלאסית: מערך נתוני Iris
10	2.5 למידה מפוקחת מול למידה לא-מפוקחת
10	2.6 האתגר הקומבינטורי
11	2.7 קללת המימד (Curse of Dimensionality)
12	3 גישה פרמטרית לקלסטרינג: ממודלים הסתברותיים לאלגוריתמים
12	3.1 המעבר מגיאומטריה להסתברות
12	3.2 מודלים גנרטיביים: מנקודות תצפית להתפלגויות
12	3.3 Maximum Likelihood Estimation: חיפוש הפרמטרים האופטימליים
13	3.4 התפלגות נורמלית: הלב הפועם של קלסטרינג פרמטרי
13	3.5 תערובות גאוסיאניות: יסוד המודל הפרמטרי
14	3.6 MLE עבור תערובת גאוסיאנית: האתגר
14	3.7 הדרך מ-GMM ל-K-Means: הנחות מפשטות
15	3.8 הקשר למרחק מהלנוביס
15	3.9 הכללה: מ-K-Means חזרה ל-Gaussian Mixture Model, GMM
16	3.10 סיכום ומבט קדימה
17	4 אלגוריתם K-Means: אלגנטיות אלגוריתמית ופשטות מתמטית
17	4.1 מהתיאוריה אל המעשה
17	4.2 פונקציית המטרה: פיזור תוך-קלסטרי
18	4.3 האלגוריתם האיטרטיבי: ריקוד בין שתי משימות
19	4.4 הוכחת התכנסות: למה זה עובד?
20	4.5 פיזור בין-קלסטרי: מדד לאיכות
20	4.6 מורכבות חישובית וביצועים
21	4.7 דוגמה מספרית פשוטה
23	5 יישום מעשי של אלגוריתם K-Means
23	5.1 מהתיאוריה למעשה: האתגר היישומי
23	5.2 אתחול מרכזי הקלסטרים: הצעד הקריטי
23	5.3 אלגוריתם K-Means++: אתחול חכם
25	5.4 אלגוריתם K-Means המלא: יישום מעשי

25	קריטריוני עצירה: מתי לעצור?	5.5
25	טיפול בקלסטרים ריקים	5.6
25	דוגמה מעשית: סימולציה עם נתונים סינתטיים	5.7
28	שימוש ב-scikit-learn: הספרייה הסטנדרטית	5.8
28	סיכום והמלצות מעשיות	5.9
31	מגבלות אלגוריתם K-Means: כאשר הפשטות הופכת למכשול	6
31	מחיר האלגנטיות	6.1
31	הנחת הקמירות: כאשר הגיאומטריה מכשילה	6.2
31	דוגמה קלאסית: בעיית המעגלים המרוכזים	6.3
32	תלות באתחול: רגישות לתנאים התחלתיים	6.4
33	בעיית בחירת K : כמה קלסטרים באמת?	6.5
33	רגישות לחרגים	6.6
34	הנחת שווי גודל ושווי צפיפות	6.7
34	קושי במרחבים רב-מימדיים	6.8
34	סיכום: מתי לא להשתמש ב-K-Means?	6.9
36	אלגוריתם K-Medoids: קלסטרינג עמיד בפני חריגים	7
36	מגבלת K-Means: רגישות לחרגים	7.1
36	הפתרון: Medoid במקום Centroid	7.2
37	אלגוריתם PAM: Partitioning Around Medoids	7.3
38	יישום PAM ב-Python	7.4
39	דוגמה מעשית: קלסטרינג מדינות	7.5
39	שימוש ב-scikit-learn-extra	7.6
39	השוואה: K-Means לעומת K-Medoids	7.7
39	מתי להשתמש ב-K-Medoids?	7.8
41	וריאציות של K-Medoids	7.9
42	סיכום: K-Medoids כחלופה עמידה	7.10
43	ניתוח Silhouette: מדידת איכות הקלסטרינג	8
43	השאלה המרכזית: האם הקלסטרינג טוב?	8.1
43	הגדרה מתמטית: מדד ה-Silhouette	8.2
44	פרשנות המקדם: מה אומר כל ערך?	8.3
44	שימוש במדד לבחירת K האופטימלי	8.4
45	דוגמה מספרית מפורטת	8.5
46	מורכבות חישובית	8.6
46	ויזואליזציה: תרשים Silhouette	8.7
47	יתרונות וחסרונות של מדד Silhouette	8.8
47	השוואה לשיטות אחרות	8.9
47	סיכום: מתי להשתמש ב-Silhouette?	8.10

48	9	הקשר ללמידת מכונה מודרנית: K-Means בעידן הרשתות העמוקות
48	9.1	מהיכן באנו ולאן אנו הולכים
48	9.2	K-Means כשלב קדם-עיבוד: Feature Learning
49	9.3	K-Means ורשתות נוירונים: קשרים מפתיעים
49	9.4	Vector Quantization ו-VQ-VAE
51	9.5	K-Means ב-Self-Supervised Learning
51	9.6	K-Means לאתחול רשתות נוירונים
53	9.7	K-Means ב-Embedding Spaces
53	9.8	אתגרים ופתרונות מודרניים
56	9.9	סיכום: K-Means בני אלמוות
57	10	יישומים מעשיים של K-Means: מהתיאוריה לתעשייה
57	10.1	מהמעבדה אל העולם האמיתי
57	10.2	פילוח לקוחות: האם אתה פרסונה או קלסטר?
58	10.3	דחיסת תמונות: פיקסלים כקלסטרים
59	10.4	זיהוי חריגות: הנורמלי מול החשוד
59	10.5	ניתוח מסמכים: מילים כווקטורים
60	10.6	ביואינפורמטיקה: גנים וחלבונים
61	10.7	המלצות: מה שאחרים אהבו
61	10.8	עיבוד תמונה רפואית: איתור גידולים
61	10.9	למידה עמוקה: K-Means כאתחול
62	10.10	מגבלות ביישומים מעשיים
62	10.11	סיכום: מדוע K-Means ממשיך לשלוט?
63	11	סיכום: K-Means – מסע מהתיאוריה למעשה
63	11.1	המסע שעברנו
63	11.2	מה למדנו? תובנות מרכזיות
64	11.3	טבלת השוואה מקיפה: אלגוריתמי קלסטרינג
64	11.4	מתי להשתמש ב-K-Means? מדריך למעשה
66	11.5	תהליך עבודה מומלץ עם K-Means
66	11.6	כיוונים עתידיים ומחקר פתוח
66	11.7	סיום: הלקח המרכזי
69	12	נספחים: פיתוחים מתמטיים ומעשיים
69	12.1	נספח א': הוכחה מפורטת של התכנסות K-Means
70	12.2	נספח ב': פיתוח נוסחת ה-rettacS Within-Between
72	12.3	נספח ג': מורכבות חישובית מפורטת
73	12.4	נספח ד': וריאציות של K-Means
74	12.5	נספח ה': טבלת השוואה - אלגוריתמי קלסטרינג
74	12.6	נספח ו': נוסחאות מרכזיות - סיכום

75		נספח ז': משאבים נוספים ללמידה	12.7
76		נספח ח': סימונים ומוסכמות	12.8
77		נספח ט': יישום מלא של אלגוריתם PAM	12.9
79		סיכום הנספחים	12.10

13 English References

80

תקציר

בשעה שהאנושות עוברת אל עידן שבו מכונות לומדות לחשוב, מתעוררת שאלה קדומה: כיצד ניתן לארגן את העולם הכאוטי שסביבנו למבנים מובנים? האם קיימות קבוצות טבעיות של אובייקטים, או שמא האדם הוא זה שכופה עליהם סדר מלאכותי? אלגוריתם K-Means, שפותח בשנות החמישים של המאה העשרים, מייצג ניסיון מתמטי מרתק לענות על שאלה זו. במהותו, זהו כלי מובהק של למידה לא-מפוקחת (Unsupervised Learning), המבקש להטיל סדר פנימי על נתונים גולמיים, בניגוד למודלים דומים דוגמת K-Nearest Neighbors (KNN), המשתמשים בתוויות קיימות לטובת חיזוי. בפרק זה נצא למסע אינטלקטואלי מהיסודות התיאורטיים של למידה לא-מפוקחת, דרך הבסיס המתמטי של אומדן נראות מקסימלית (Maximum Likelihood Estimation), ועד ליישומים מעשיים המשנים את פני התעשייה המודרנית.

נחקור את היופי הגלום בפשטות האלגוריתמית של K-Means, אך גם את מגבלותיו ואת המחיר שאנו משלמים על האלגנטיות המתמטית. נבחן כיצד אלגוריתם זה, שנולד בעידן שבו מחשבים מילאו חדרים שלמים, הפך לאבן יסוד בארכיטקטורות הלמידה העמוקה המודרניות.

2 בעיית הקלסטרינג: מסע אל עולם הלמידה הלא-מפוקחת

2.1 פילוסופיה של קטגוריזציה

האם העולם מחולק באופן טבעי לקטגוריות, או שמא אנו, בני האדם, אלו שכופים עליו סדר מלאכותי? שאלה זו, הנשאלת מאז שחר הפילוסופיה, מקבלת משנה תוקף בעידן הלמידה החישובית. כאשר אפלטון חילק את העולם לצורות אידיאליות (Forms), הוא למעשה ביצע קלסטרינג פילוסופי – ארגון של תצפיות אינסופיות למספר סופי של קטגוריות מופשטות. בעיית הקלסטרינג (Clustering Problem) היא אחת השאלות המרכזיות בלמידה לא-מפוקחת (Unsupervised Learning). בניגוד ללמידה מפוקחת (Supervised Learning), שבה אנו יודעים מראש את התוויות של הנתונים, בקלסטרינג אנו מבקשים לגלות מבנה נסתר בנתונים ללא כל הנחיה חיצונית. זהו, במובן מסוים, תהליך של גילוי ידע – Knowledge Discovery – שבו האלגוריתם עצמו מחליט כיצד לארגן את המידע.

2.2 הגדרה מתמטית פורמלית

נניח שיש בידינו אוסף של n תצפיות במרחב d -מימדי:

$$(1) \quad \mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d$$

מטרתנו היא לחלק את האוסף $\mathcal{X}$ ל- k קבוצות זרות (קלסטרים):

$$(2) \quad \mathcal{C} = \{C_1, C_2, \dots, C_k\}$$

כך שמתקיימים התנאים הבאים:

תנאי כיסוי מלא: כל תצפית שייכת לפחות לקלסטר אחד:

$$(3) \quad \bigcup_{j=1}^k C_j = \mathcal{X}$$

תנאי זרות (במקרה של Hard Clustering): כל תצפית שייכת לקלסטר אחד בלבד:

$$(4) \quad C_i \cap C_j = \emptyset, \quad \forall i \neq j$$

אולם הגדרה זו עדיין לא מספקת. היא לא מספרת לנו איזו חלוקה היא "טובה". כאן נכנס לתמונה המושג המרכזי: **דמיון פנים-קלסטרי** (Intra-cluster Similarity) לעומת **דמיון בין-קלסטרי** (Inter-cluster Similarity). אנו מבקשים חלוקה שבה:

- תצפיות **בתוך** אותו קלסטר דומות זו לזו (דמיון גבוה)

- תצפיות **בין** קלסטרים שונים שונות זו מזו (דמיון נמוך)

2.3 מטריקות מרחק במרחבים מטריים

כדי לכמת את המושג "דמיון", עלינו להגדיר מטריקת מרחק (Distance Metric). מרחב מטרי $(\mathcal{X}, d)$ מורכב מקבוצה $\mathcal{X}$ ומפונקציית מרחק $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$ המקיימת את האקסיומות הבאות עבור כל $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathcal{X}$:

1. **אי-שליליות:** $d(\mathbf{x}, \mathbf{y}) \geq 0$, כאשר $d(\mathbf{x}, \mathbf{y}) = 0$ אם ורק אם $\mathbf{x} = \mathbf{y}$

2. **סימטריה:** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$

3. **אי-שוויון המשולש:** $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$

המטריקה הנפוצה ביותר היא **מרחק אוקלידי** (Euclidean Distance):

$$(5) \quad d_2(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

מטריקה זו נובעת מהמכפלה הפנימית הסטנדרטית ומגדירה גיאומטריה היפרבולית. אולם קיימות מטריקות נוספות, כגון:

מרחק מנהטן (Manhattan Distance, נורמת L_1):

$$(6) \quad d_1(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_1 = \sum_{i=1}^d |x_i - y_i|$$

מרחק מינקובסקי כללי (Minkowski Distance):

$$(7) \quad d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$$

כאשר $p \rightarrow \infty$, מקבלים את **מרחק צ'בישב** (Chebyshev Distance):

$$(8) \quad d_\infty(\mathbf{x}, \mathbf{y}) = \max_{i=1, \dots, d} |x_i - y_i|$$

בחירת המטריקה המתאימה היא בעלת חשיבות עליונה ותלויה בטבע הנתונים. למשל, בניתוח טקסטים נפוץ להשתמש ב**דמיון קוסינוס** (Cosine Similarity), המודד את הזווית בין וקטורים ולא את המרחק האבסולוטי ביניהם.

2.4 דוגמה קלאסית: מערך נתוני Iris

אחד המערכים המפורסמים ביותר בלמידת מכונה הוא מערך הנתונים של רונלד פישר מ-1936 [1]. מערך Iris מכיל 150 מדידות של פרחי אירוס משלושה מינים: Iris, Iris setosa, Iris versicolor, ו-Iris virginica. לכל פרח נמדדו ארבעה תכונות:

- אורך עלה-גביע (Sepal Length)

- רוחב עלה-גביע (Sepal Width)

- אורך עלה-כותרת (Petal Length)

- רוחב עלה-כותרת (Petal Width)

כלומר, $\mathbf{x}_i \in \mathbb{R}^4$ עבור $i = 1, \dots, 150$. במקרה זה, אנו יודעים את התוויות האמיתיות (שלושת המינים), אך האתגר המעניין הוא: האם אלגוריתם קלסטרינג יכול לגלות את החלוקה הטבעית הזו ללא כל מידע מוקדם? דוגמה זו ממחישה את הפרדוקס המרכזי בלמידה לא-מפוקחת: אנו משתמשים בתוויות רק לאחר מעשה, לצורך הערכת ביצועי האלגוריתם, אך האלגוריתם עצמו אינו רואה אותן במהלך הלמידה.

2.5 למידה מפוקחת מול למידה לא-מפוקחת

כדי להבין לעומק את אופי בעיית הקלסטרינג, חשוב להבחין בין שני פרדיגמות הלמידה: **למידה מפוקחת (Supervised Learning):**

- קיים אוסף אימון $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ כאשר y_i הן תוויות ידועות

- המטרה: ללמוד פונקציה $f: \mathcal{X} \rightarrow \mathcal{Y}$ שמנבאת תוויות לנתונים חדשים

- דוגמאות: סיווג (Classification), רגרסיה (Regression)

למידה לא-מפוקחת (Unsupervised Learning):

- קיים רק אוסף $\{\mathbf{x}_i\}_{i=1}^n$ ללא תוויות

- המטרה: לגלות מבנה, דפוסים, או ייצוג חבוי בנתונים

- דוגמאות: קלסטרינג, הפחתת מימדים (Dimensionality Reduction), איתור אנומליות

הבדל מהותי נוסף: בלמידה מפוקחת קיים קריטריון אובייקטיבי ברור להצלחה (דיוק על קבוצת מבחן), ואילו בקלסטרינג אין הגדרה אוניברסלית למה זה "קלסטר טוב". זו אחת האתגרים העמוקים ביותר בתחום.

2.6 האתגר הקומבינטורי

מספר החלוקות האפשריות של n תצפיות ל- k קלסטרים לא-ריקים נתון על ידי **מספר סטירלינג מסוג שני** (Stirling Number of the Second Kind):

$$(9) \quad S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

עבור $n = 100$ ו- $k = 5$, למשל, מספר האפשרויות הוא בסדר גודל של 10^{68} – גדול בהרבה ממספר האטומים ביקום הנצפה! חיפוש ממצה על כל האפשרויות הוא בלתי אפשרי, ולכן נזקקים לאלגוריתמים היוריסטיים יעילים. זו בדיוק הסיבה שאלגוריתם K-Means, למרות פשטותו, הוא כה חשוב: הוא מספק קירוב יעיל חישובית לבעיה קומבינטורית בלתי פתירה.

2.7 קללת המימד (Curse of Dimensionality)

ככל שמימד המרחב d גדל, תופעה מוזרה מתרחשת: כל הנקודות נראות רחוקות זו מזו באותה מידה. זוהי "קללת הממד", שהוכחה לראשונה על ידי ריצ'רד בלמן [2]. במרחבים רב-מימדיים, היחס בין המרחק המינימלי למרחק המקסימלי בין נקודות שואף ל-1:

$$(10) \quad \lim_{d \rightarrow \infty} \frac{\text{dist}_{\min}}{\text{dist}_{\max}} \rightarrow 1$$

משמעות הדבר: מושג "קרבה" הופך לא-מובחן, וקלסטרינג הופך למאתגר ביותר. לכן, בפועל, נהוג לבצע הפחתת מימדים (Dimensionality Reduction) כגון PCA לפני יישום קלסטרינג במרחבים רב-מימדיים.

3 גישה פרמטרית לקלסטרינג: ממודלים הסתברותיים לאלגוריתמים

3.1 המעבר מגיאומטריה להסתברות

בפרק הקודם ראינו כיצד ניתן להגדיר את בעיית הקלסטרינג כבעיית אופטימיזציה גיאומטרית במרחב מטרי. כעת, נעבור לגישה מהפכנית שמביאה את עולם ההסתברות והסטטיסטיקה לתמונה – **הגישה הפרמטרית** (Parametric Approach).

במקום לחשוב על קלסטרים כאזורים במרחב, נדמיין כל קלסטר כ**התפלגות הסתברותית** (Probability Distribution) עם פרמטרים ספציפיים. גישה זו, שמקורה בעבודתו החלוצית של רונלד פישר בשנות ה-1920 [3], מאפשרת לנו להשתמש בכלים חזקים מתורת ההסתברות והסטטיסטיקה ליצירת אלגוריתמי קלסטרינג יעילים ואלגנטיים.

3.2 מודלים גנרטיביים: מנקודות תצפית להתפלגויות

מודל גנרטיבי (Generative Model) הוא מודל מתמטי המתאר כיצד הנתונים שלנו נוצרו. הרעיון המרכזי פשוט אך עוצמתי:

"כל נקודת מידע נוצרה מתוך אחת מ- K התפלגויות הסתברותיות, כאשר כל התפלגות מייצגת קלסטר אחד."

פורמלית, נניח שיש לנו K התפלגויות, כל אחת מאופיינת בפרמטרים θ_k . כל נקודת תצפית x_i נוצרה מתוך אחת ההתפלגויות הללו לפי ההסתברות $p(x_i|\theta_k)$. התהליך הגנרטיבי נראה כך:

1. בחר קלסטר k לפי ההסתברות π_k (משקל הקלסטר במערך הנתונים)

2. דגום נקודה x_i מההתפלגות $p(x|\theta_k)$

3. חזור על תהליך זה n פעמים ליצירת מערך הנתונים המלא

3.3 Maximum Likelihood Estimation: חיפוש הפרמטרים האופטימליים

אומדן נראות מקסימלית (Maximum Likelihood Estimation, MLE) הוא עקרון יסודי בסטטיסטיקה, שפותח על ידי רונלד פישר ב-1912 [4]. העיקרון פשוט: בהינתן מערך נתונים, נמצא את הפרמטרים שמגדילים את ההסתברות לקבל בדיוק את הנתונים שראינו. **פונקציית הנראות** (Likelihood Function) מוגדרת כך:

$$(11) \quad \mathcal{L}(\theta|X) = \prod_{i=1}^n p(x_i|\theta)$$

כאשר $X = \{x_1, x_2, \dots, x_n\}$ הוא מערך הנתונים, ו- θ הם הפרמטרים של המודל.

מכיוון שמכפלות גדולות מסובכות לעבודה, משתמשים ב**לוג-נראות** (Log-Likelihood):

$$(12) \quad \ell(\theta|X) = \log \mathcal{L}(\theta|X) = \sum_{i=1}^n \log p(x_i|\theta)$$

המטרה שלנו: למצוא $\hat{\theta}$ שממקסם את פונקציית הלוג-נראות:

$$(13) \quad \hat{\theta}_{MLE} = \arg \max_{\theta} \ell(\theta|X)$$

3.4 התפלגות נורמלית: הלב הפועם של קלסטרינג פרמטרי

ההתפלגות הנורמלית (Normal/Gaussian Distribution) היא אבן היסוד של רוב שיטות הקלסטרינג הפרמטריות, בזכות תכונותיה המתמטיות האלגנטיות והמשמעות המעשית שלה.

ההגדרה המתמטית:

התפלגות נורמלית חד-משתנית (Univariate) עם תוחלת μ וסטית σ :

$$(14) \quad p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

התפלגות נורמלית רב-משתנית (Multivariate) במימד d עם וקטור תוחלת μ ומטריצת שונות-משותפת Σ :

$$(15) \quad p(\mathbf{x}|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1}(\mathbf{x}-\mu)\right)$$

מדוע ההתפלגות הנורמלית כל כך חשובה?

1. **משפט הגבול המרכזי** (Central Limit Theorem): סכום של משתנים מקריים רבים נוטה להתפלגות נורמלית, ללא תלות בהתפלגויות המקוריות

2. **פרמטריזציה פשוטה**: שני פרמטרים בלבד (μ, σ^2) מגדירים את ההתפלגות לחלוטין

3. **סימטריה ואלגנטיות מתמטית**: נגזרות ואינטגרלים של פונקציות גאוסיאניות נותנות פונקציות גאוסיאניות

4. **תכונת המקסימום אנטרופיה**: בהינתן תוחלת ושונות, ההתפלגות הנורמלית היא זו עם האנטרופיה המקסימלית

3.5 תערובות גאוסיאניות: יסוד המודל הפרמטרי

מודל תערובת גאוסיאנית (Gaussian Mixture Model, GMM) הוא הלב של הגישה הפרמטרית לקלסטרינג. הרעיון: מערך הנתונים נוצר מתערובת של K התפלגויות נורמליות, כל אחת מייצגת קלסטר.

ההגדרה הפורמלית:

ההסתברות שנקודה $\mathbf{x}$ נוצרה מהמודל:

$$(16) \quad p(\mathbf{x}|\Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k)$$

כאשר:

- π_k - משקל הקלסטר ה- k , כך ש- $\sum_{k=1}^K \pi_k = 1$

- $\boldsymbol{\mu}_k$ - וקטור התוחלת של הקלסטר ה- k

- Σ_k - מטריצת השונות-משותפת של הקלסטר ה- k

- $\Theta = \{\pi_k, \boldsymbol{\mu}_k, \Sigma_k\}_{k=1}^K$ - מכלול הפרמטרים

משמעות גיאומטרית:

כל קלסטר הוא "ענף" אליפטי של נקודות במרחב $\mathbb{R}^d$, כאשר:

- $\boldsymbol{\mu}_k$ קובע את מרכז הענף

- Σ_k קובע את הצורה והכיוון של האליפסה

- π_k קובע את "צפיפות" הנקודות בענף

3.6 MLE עבור תערובת גאוסיאנית: האתגר

חישוב MLE עבור GMM אינו טריוויאלי. פונקציית הלוג-נראות:

$$(17) \quad \ell(\Theta|X) = \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_i|\boldsymbol{\mu}_k, \Sigma_k) \right)$$

הבעיה: הלוגריתם של סכום אינו ניתן לפתרון אנליטי פשוט. אין נוסחה סגורה ל- $\hat{\Theta}_{MLE}$.

הפתרון: נדרשת שיטה איטרטיבית. כאן נכנס לתמונה **אלגוריתם Expectation-**

Maximization (EM), שפותח על ידי דמפסטר, לירד ורובין ב-1977 [5].

אבל... יש דרך פשוטה יותר לבעיית הקלסטרינג: אם נניח הנחות מפשטות על מבנה

GMM, נוכל להגיע לאלגוריתם **K-Means**, שהוא למעשה מקרה פרטי של EM עבור GMM.

3.7 הדרך מ-GMM ל-K-Means: הנחות מפשטות

כדי להגיע לאלגוריתם K-Means, נבצע שלוש הנחות מפשטות על מודל GMM:

הנחה 1: קלסטרים שווי-משקל

$$(18) \quad \pi_k = \frac{1}{K} \quad \forall k$$

כל קלסטר מכיל אותו מספר נקודות בממוצע.

הנחה 2: מטריצת שונות-משותפת ספרית

$$(19) \quad \Sigma_k = \sigma^2 \mathbf{I} \quad \forall k$$

כל קלסטר הוא "כדור" (לא אליפסה), עם אותה שונות בכל הכיוונים.

הנחה 3: שונות שואפת לאפס

$$(20) \quad \sigma^2 \rightarrow 0$$

כל נקודה שייכת באופן דטרמיניסטי לקלסטר אחד בלבד (לא קיימת "חפיפה" בין קלסטרים).

התוצאה המפתיעה:

תחת הנחות אלו, בעיית MLE עבור GMM מצטמצמת ל:

$$(21) \quad \min_{\{\mu_k\}_{k=1}^K} \sum_{i=1}^n \min_k \|\mathbf{x}_i - \mu_k\|^2$$

זוהי בדיוק פונקציית המטרה של K-Means!

במילים אחרות: K-Means הוא אלגוריתם למציאת MLE של מודל GMM תחת הנחות מפשטות מסוימות.

3.8 הקשר למרחק מהלנוביס

כדי להבין טוב יותר את הקשר בין GMM ל-K-Means, נזכיר את **מרחק מהלנוביס** (Maha-lanobis Distance), שפותח על ידי פרסנטה צ'נדרה מהלנוביס ב-1936 [6]. עבור נקודה $\mathbf{x}$ וקלסטר עם תוחלת μ ומטריצת שונות Σ :

$$(22) \quad d_M(\mathbf{x}, \mu) = \sqrt{(\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)}$$

כאשר $\Sigma = \sigma^2 \mathbf{I}$ (הנחה 2 שלנו), המרחק הופך למרחק אוקלידי:

$$(23) \quad d_M(\mathbf{x}, \mu) = \frac{1}{\sigma} \|\mathbf{x} - \mu\|_2$$

זוהי הסיבה ש-K-Means משתמש במרחק אוקלידי – זה המרחק הטבעי תחת ההנחות הפרמטריות שלנו.

3.9 הכללה: מ-K-Means חזרה ל-Gaussian Mixture Model

ההבנה הפרמטרית מאפשרת לנו להכליל את K-Means בדרכים שונות:

1. **אלגוריתם EM עבור GMM מלא:** ללא הנחות מפשטות, מאפשר קלסטרים אליפטיים ובגדלים שונים

2. **Fuzzy C-Means**: הרפיה של הנחה 3, מאפשר שיוך "רך" (Soft Assignment) של נקודות לקלסטרים

3. **K-Medoids**: שימוש בהתפלגויות אחרות (לא נורמליות) או מטריקות מרחק אחרות

3.10 סיכום ומבט קדימה

בפרק זה ראינו כיצד גישה פרמטרית, המבוססת על מודלים הסתברותיים, מובילה באופן טבעי לאלגוריתם K-Means. העבודה שלנו כעת היא כפולה:

1. להבין את **האלגוריתם עצמו** – כיצד הוא פועל, מה היא המורכבות החישובית, ומהן התכונות שלו

2. להבין את **ההשלכות המעשיות** – מתי K-Means מצליח ומתי הוא נכשל, וכיצד להתגבר על מגבלותיו

בפרק הבא נפרט את האלגוריתם עצמו, נוכיח את ההתכנסות שלו, וננתח את המורכבות החישובית.

4 אלגוריתם K-Means: אלגנטיות אלגוריתמית ופשטות מתמטית

4.1 מהתיאוריה אל המעשה

לאחר שהבנו את הבסיס התיאורטי של למידה לא-מפוקחת ואת הגישה הפרמטרית למידול קלסטרים, הגיע הזמן לבחון את אחד האלגוריתמים האלגנטיים ביותר בלמידת מכונה: K-Means. אלגוריתם זה, שפותח באופן עצמאי על ידי Stuart Lloyd בשנת 1957 [7] ועל ידי Edward W. Forgy בשנת 1965 [8], מייצג פתרון יפהפה לבעיה מתמטית מורכבת.

היופי של K-Means טמון בפשטות: במקום לפתור אופטימיזציה קומבינטורית בלתי-פתירה, האלגוריתם מציע אסטרטגיה איטרטיבית שמתכנסת למינימום לוקלי. זוהי תובנה עמוקה - לעיתים, פתרון "מספיק טוב" שניתן לחישוב הוא עדיף על פתרון "אופטימלי" שלא ניתן לחישוב בפועל.

בפרק זה נחקור את המבנה המתמטי של האלגוריתם, נוכיח את התכנסותו, ונבחן את המורכבות החישובית שלו. נראה כיצד אלגוריתם פשוט זה הפך לאחד מאבני היסוד של למידת מכונה מודרנית.

4.2 פונקציית המטרה: פיזור תוך-קלסטרי

הרעיון המרכזי של K-Means הוא למזער את הפיזור התוך-קלסטרי (Within-Cluster Scatter, WCS) או בקיצור WCS). פונקציה זו מודדת עד כמה נקודות הנתונים "קרובות" למרכזי הקלסטרים שלהן.

נגדיר באופן פורמלי. נניח שיש לנו מערך נתונים $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ כאשר $\mathbf{x}_i \in \mathbb{R}^d$, ואנו רוצים לחלק אותו ל- K קלסטרים. פונקציית המטרה היא:

$$(24) \quad J(\mathcal{C}, \boldsymbol{\mu}) = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

כאשר:

- K - מספר הקלסטרים (פרמטר נתון מראש)

- $\mathcal{C} = \{C_1, \dots, C_K\}$ - חלוקה של הנתונים לקלסטרים

- C_k - הקלסטר ה- k , קבוצת האינדקסים של נקודות השייכות לקלסטר זה

- $\mathbf{x}_i$ - נקודת נתונים במרחב $\mathbb{R}^d$

- $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$ - מרכז הקלסטר ה- k (centroid)

- $\|\mathbf{x} - \boldsymbol{\mu}\|^2 = \sum_{j=1}^d (x_j - \mu_j)^2$ - ריבוע מרחק אוקלידי (squared Euclidean distance)

מדוע ריבועים? ישנן מספר סיבות מתמטיות ותיאורטיות:

- **גזירות:** פונקציית הריבוע גזירה באופן רציף, מה שמאפשר שימוש בכלים של חשבון דיפרנציאלי.

- **קמירות:** הפונקציה קמורה ביחס למרכזים μ_k (אם כי לא ביחס להקצאות).

- **קשר לנורמליות:** השימוש בריבועים תואם את ההנחה שהנתונים מגיעים מהתפלגות נורמלית.

- **פשטות חישובית:** חישוב ריבוע המרחק מהיר וקל ליישום.

4.3 האלגוריתם האיטרטיבי: ריקוד בין שתי משימות

אלגוריתם K-Means הוא למעשה שיטה איטרטיבית המתחלפת בין שני שלבים משלימים. אלגוריתם זה מכונה גם Lloyd's Algorithm על שם ממציאו, ובספרות מקובל לקרוא לו גם K-Means Clustering או בקיצור K-Means.

אתחול (Initialization):

בחר K מרכזי קלסטרים התחלתיים $\mu_1^{(0)}, \dots, \mu_K^{(0)}$. קיימות מספר אסטרטגיות אתחול:

- **אקראי:** בחר K נקודות באופן אקראי מתוך מערך הנתונים.

- **K-Means++:** שיטת אתחול חכמה יותר שמפזרת את המרכזים ההתחלתיים [9].

- **ידני:** במקרים מסוימים ניתן לבחור מרכזים התחלתיים על סמך ידע תחומי.

איטרציה t (Iteration):

שלב 1 - הקצאה (Assignment Step):

הקצה כל נקודה x_i לקלסטר הקרוב ביותר (במובן של מרחק אוקלידי):

$$(25) \quad c_i^{(t)} = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k^{(t-1)}\|^2$$

כלומר, $c_i^{(t)}$ הוא האינדקס של הקלסטר שאליו משויכת נקודה i באיטרציה t .

שלב 2 - עדכון (Update Step):

חשב מחדש את מרכזי הקלסטרים כממוצע חשבוני של הנקודות המוקצות אליהם:

$$(26) \quad \mu_k^{(t)} = \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} x_i$$

כאשר $C_k^{(t)} = \{i : c_i^{(t)} = k\}$ הוא קבוצת האינדקסים של נקודות השייכות לקלסטר k באיטרציה t .

תנאי עצירה (Stopping Criterion):

חזור על האיטרציה עד שמתקיים אחד מהתנאים הבאים:

- אין שינוי בהקצאות: $c_i^{(t)} = c_i^{(t-1)}$ לכל i

- אין שינוי במרכזים: $\mu_k^{(t)} = \mu_k^{(t-1)}$ לכל k
- השינוי בפונקציית המטרה קטן מאוד: $|J^{(t)} - J^{(t-1)}| < \epsilon$
- הגענו למספר איטרציות מקסימלי מוגדר מראש

4.4 הוכחת התכנסות: למה זה עובד?

אחד היתרונות המרכזיים של אלגוריתם K-Means הוא שניתן להוכיח באופן ריגורוזי שהוא מתכנס. נוכיח את המשפט הבא:

משפט (התכנסות K-Means): אלגוריתם K-Means מתכנס למינימום לוקלי של פונקציית המטרה J .

הוכחה:

נוכיח שבכל איטרציה, פונקציית המטרה J קטנה (או לכל היותר נשארת קבועה):

(1) שלב ההקצאה מקטין את J :

בשלב ההקצאה, אנו בוחרים עבור כל נקודה $\mathbf{x}_i$ את הקלסטר שמזער את המרחק:

$$(27) \quad c_i^{(t)} = \arg \min_k \|\mathbf{x}_i - \mu_k^{(t-1)}\|^2$$

כלומר, לכל נקודה אנו בוחרים את ההקצאה ה"טובה ביותר" בהינתן המרכזים הנוכחיים. לכן:

$$(28) \quad J(\mathcal{C}^{(t)}, \mu^{(t-1)}) \leq J(\mathcal{C}^{(t-1)}, \mu^{(t-1)})$$

(2) שלב העדכון מקטין את J :

בשלב העדכון, אנו מחפשים עבור כל קלסטר k את המרכז μ_k שמזער את הסכום:

$$(29) \quad \sum_{i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

נגזור לפי μ_k ונשווה לאפס:

$$(30) \quad \frac{\partial}{\partial \mu_k} \sum_{i \in C_k} \|\mathbf{x}_i - \mu_k\|^2 = -2 \sum_{i \in C_k} (\mathbf{x}_i - \mu_k) = 0$$

מכאן נקבל:

$$(31) \quad \mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

זהו בדיוק הממוצע החשבוני! לכן:

$$(32) \quad J(\mathcal{C}^{(t)}, \mu^{(t)}) \leq J(\mathcal{C}^{(t)}, \mu^{(t-1)})$$

(3) סיכום:

בכל איטרציה מתקיים:

$$(33) \quad J^{(t)} = J(\mathcal{C}^{(t)}, \mu^{(t)}) \leq J(\mathcal{C}^{(t)}, \mu^{(t-1)}) \leq J(\mathcal{C}^{(t-1)}, \mu^{(t-1)}) = J^{(t-1)}$$

כלומר, J יורדת מונוטונית. מכיוון ש- J חסומה מלמטה (אפס), היא מתכנסת. בנוסף, מכיוון שיש רק מספר סופי של הקצאות אפשריות (K^n), האלגוריתם חייב להגיע לנקודת שיווי משקל לאחר מספר סופי של איטרציות. $\square$

אזהרה קריטית: ההתכנסות היא למינימום **לוקלי**, לא גלובלי! לכן תוצאות K-Means תלויות באתחול, ובפועל נהוג להריץ את האלגוריתם מספר פעמים עם אתחולים שונים ולבחור את הפתרון הטוב ביותר.

4.5 פיזור בין-קלסטרי: מדד לאיכות

בנוסף לפיזור התוך-קלסטרי, ניתן להגדיר את **הפיזור הבין-קלסטרי** (Between-Cluster Scatter, BCS) או בקיצור BCS):

$$(34) \quad B = \sum_{k=1}^K |C_k| \cdot \|\mu_k - \mu\|^2$$

כאשר $\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ הוא מרכז הכובד הכללי של כל הנתונים. מדד זה מודד עד כמה מרכזי הקלסטרים "מפוזרים" זה מזה. קלסטרינג טוב אמור להיות בעל פיזור תוך-קלסטרי נמוך (נקודות קרובות למרכזים) ופיזור בין-קלסטרי גבוה (קלסטרים רחוקים זה מזה).

תובנה מתמטית: הסכום הכולל קבוע:

$$(35) \quad \text{TSS} = J + B = \sum_{i=1}^n \|\mathbf{x}_i - \mu\|^2 = \text{const}$$

כאשר TSS (Total Sum of Squares) הוא הפיזור הכולל. לכן:

$$(36) \quad \min J \Leftrightarrow \max B$$

זהו עיקרון יסודי: מזעור הפיזור התוך-קלסטרי שקול למקסום הפיזור הבין-קלסטרי.

4.6 מורכבות חישובית וביצועים

מורכבות זמן לאיטרציה:

בכל איטרציה, אנו מבצעים:

- **שלב הקצאה:** לכל אחת מ- n הנקודות, מחשבים מרחק ל- K מרכזים. כל חישוב מרחק דורש $O(d)$ פעולות. סך הכל: $O(nKd)$.

- **שלב עדכון:** סכימת n נקודות ב- d מימדים וחלוקה ב- K קלסטרים: $O(nd + K)$.

סה"כ לאיטרציה: $O(nKd)$.

מספר איטרציות:

במקרה הגרוע ביותר, מספר האיטרציות יכול להיות אקספוננציאלי ב- n [10]. עם זאת, בפועל האלגוריתם מתכנס בדרך כלל לאחר מספר קטן של איטרציות (עשרות לכל היותר).

מורכבות כוללת:

אם מספר האיטרציות הוא T , המורכבות הכוללת היא $O(TnKd)$. בפועל, זהו אלגוריתם יעיל מאוד עבור מערכי נתונים גדולים.
יתרון מעשי: K-Means הוא אחד האלגוריתמים היעילים ביותר לקלסטרינג, ולכן נמצא בשימוש נרחב בתעשייה ובאקדמיה.

4.7 דוגמה מספרית פשוטה

נדגים את האלגוריתם על מערך נתונים דו-מימדי זעיר. יהי:
נתונים:

$$\mathbf{x}_1 = (1, 1), \quad \mathbf{x}_2 = (2, 1), \quad \mathbf{x}_3 = (5, 5), \quad \mathbf{x}_4 = (6, 5) \quad (37)$$

אתחול: נבחר שני מרכזים התחלתיים:

$$\mu_1^{(0)} = (1, 1), \quad \mu_2^{(0)} = (5, 5) \quad (38)$$

איטרציה 1:

שלב הקצאה: נחשב מרחקים:

$$\|\mathbf{x}_1 - \mu_1\|^2 = 0 < \|\mathbf{x}_1 - \mu_2\|^2 = 32 \Rightarrow c_1 = 1 -$$

$$\|\mathbf{x}_2 - \mu_1\|^2 = 1 < \|\mathbf{x}_2 - \mu_2\|^2 = 25 \Rightarrow c_2 = 1 -$$

$$\|\mathbf{x}_3 - \mu_1\|^2 = 32 > \|\mathbf{x}_3 - \mu_2\|^2 = 0 \Rightarrow c_3 = 2 -$$

$$\|\mathbf{x}_4 - \mu_1\|^2 = 41 > \|\mathbf{x}_4 - \mu_2\|^2 = 1 \Rightarrow c_4 = 2 -$$

$$\text{לכן: } C_2 = \{3, 4\}, C_1 = \{1, 2\}.$$

שלב עדכון:

$$\mu_1^{(1)} = \frac{(1, 1) + (2, 1)}{2} = (1.5, 1) \quad (39)$$

$$\mu_2^{(1)} = \frac{(5, 5) + (6, 5)}{2} = (5.5, 5) \quad (40)$$

איטרציה 2:

שלב הקצאה: בדיקה מראה שכל הנקודות נשארות באותם קלסטרים $\Rightarrow$ התכנסות!

תוצאה סופית:

- קלסטר 1: $\{(1, 1), (2, 1)\}$ עם מרכז $(1.5, 1)$

- קלסטר 2: $\{(5, 5), (6, 5)\}$ עם מרכז $(5.5, 5)$

זוהי חלוקה אינטואיטיבית לשני קלסטרים נפרדים היטב!

5 יישום מעשי של אלגוריתם K-Means

5.1 מהתיאוריה למעשה: האתגר היישומי

בפרקים הקודמים חקרנו את היסודות התיאורטיים של קלסטרינג ואת הבסיס המתמטי של אלגוריתם K-Means. כעת נעבור לשאלה המעשית המרכזית: כיצד מיישמים את האלגוריתם בפועל? מהן ההחלטות הקריטיות שעל המפתח לקבל? ואילו מלכודות עלינו להימנע מהן? יישום אפקטיבי של K-Means דורש הבנה מעמיקה של מספר היבטים מעשיים: אתחול נכון של מרכזי הקלסטרים, בחירת מספר הקלסטרים K , טיפול בנתונים חסרים ובחריגים, והכרעה מתי האלגוריתם התכנס. כל אחד מההיבטים הללו עשוי להשפיע באופן דרמטי על איכות התוצאות.

5.2 אתחול מרכזי הקלסטרים: הצעד הקריטי

הבעיה: אלגוריתם K-Means רגיש באופן קיצוני לאתחול הראשוני של מרכזי הקלסטרים. אתחול שגוי עלול להוביל למינימום מקומי גרוע, וכתוצאה מכך לחלוקה לא-אופטימלית של הנתונים לקלסטרים.

שיטת האתחול הנאיבית: בחירה אקראית של K נקודות מתוך הנתונים כמרכזי קלסטרים ראשוניים:

הבעיה באתחול אקראי: שני מרכזים עלולים להיבחר קרוב מאוד זה לזה, מה שיגרום לקלסטרים חופפים ותוצאות לא-יציבות. פתרון אלגנטי לבעיה זו הוא שיטת K-Means++.

5.3 אלגוריתם K-Means++ : אתחול חכם

אלגוריתם K-Means++ שפותח על ידי דייוויד ארתור וסרגיי וסילביצקי ב-2007 [9], מבטיח אתחול טוב יותר על ידי בחירת מרכזים ראשוניים שמפוזרים היטב במרחב.

רעיון מרכזי: כל מרכז חדש נבחר עם הסתברות פרופורציונלית למרחק המינימלי שלו מהמרכזים שכבר נבחרו. כך, נקודות רחוקות מהמרכזים הקיימים זוכות להסתברות גבוהה יותר להיבחר.

האלגוריתם:

1. בחר מרכז ראשון μ_1 באופן אקראי אחיד מתוך X

2. לכל $j = 2, 3, \dots, K$:

(א) עבור כל נקודה x_i , חשב $D(x_i)^2$ – ריבוע המרחק למרכז הקרוב ביותר:

$$(41) \quad D(x_i) = \min_{1 \leq \ell < j} \|\mathbf{x}_i - \mu_\ell\|^2$$

(ב) בחר את המרכז הבא μ_j עם הסתברות:

$$(42) \quad P(\mathbf{x}_i) = \frac{D(\mathbf{x}_i)^2}{\sum_{m=1}^n D(\mathbf{x}_m)^2}$$

אתחול אקראי נאיבי

```
import numpy as np

def random_initialization(X, k):
    """
    מירטסלקן יזכרמלש יארקא לוחתא
    Parameters:
    X: array-like, shape (n_samples, n_features)
    סינותנה ןרעמ
    k: int
    מירטסלקה רפסמ
    Returns:
    centroids: array, shape (k, n_features)
    סינוש ארה מירטסלקה יזכרמ
    """
    n_samples = X.shape[0]
    random_indices = np.random.choice(n_samples, k, replace=False)
    centroids = X[random_indices]
    return centroids
```


יישום ב-Python:

מה השיפור? ארתור ווסילביצקי הוכיחו ש-K-Means++ מבטיח ערך מטרָה שהוא לכל היותר $O(\log K)$ פעמים גרוע יותר מהאופטימום הגלובלי, בעוד שאתחול אקראי נאיבי עלול להיות גרוע באופן שרירותי.

5.4 אלגוריתם K-Means המלא: יישום מעשי

כעת נממש את האלגוריתם המלא, כולל האתחול החכם וקריטריון עצירה:

5.5 קריטריוני עצירה: מתי לעצור?

האלגוריתם צריך לדעת מתי להפסיק את האיטרציות. ישנם מספר קריטריונים אפשריים:

1. שינוי זניח במרכזים:

$$(43) \quad \max_{j=1,\dots,K} \|\mu_j^{(t+1)} - \mu_j^{(t)}\| < \epsilon$$

2. שינוי זניח בפונקציית המטרה:

$$(44) \quad \frac{|J^{(t)} - J^{(t+1)}|}{J^{(t)}} < \epsilon$$

3. מספר מקסימלי של איטרציות: למנוע ריצה אינסופית במקרים נדירים של אי-התכנסות.

4. אין שינוי בשיוך הנקודות: אם לא נקודה אחת לא שינתה קלסטר, האלגוריתם התכנס.

ביישום שלנו משתמשים בשילוב של קריטריונים 1 ו-3.

5.6 טיפול בקלסטרים ריקים

בעיה מעשית נפוצה: במהלך האלגוריתם, קלסטר עלול להיוותר ללא נקודות. זה קורה כאשר כל הנקודות שהיו בקלסטר עברו לקלסטרים אחרים.

פתרונות אפשריים:

1. שמירת המרכז הישן: כפי שעשינו ביישום לעיל

2. בחירת נקודה חדשה: בחר את הנקודה הרחוקה ביותר ממרכז הקלסטר שלה כמרכז חדש

3. פיצול קלסטר גדול: פצל את הקלסטר הגדול ביותר לשניים

5.7 דוגמה מעשית: סימולציה עם נתונים סינתטיים

כדי להמחיש את האלגוריתם, ניצור מערך נתונים סינתטי ונריץ עליו את K-Means:

```

def kmeans_plusplus_init(X, k):
    """
    מירטסלקיזכרמלשםכחלוחתאם-K-Means++לוחתאם

    Parameters:
    -----
    X: array-like, shape (n_samples, n_features)
        מיןותנהדרעמ
    k: int
        מירטסלקהרפסמ

    Returns:
    -----
    centroids: array, shape (k, n_features)
        מיינושארמלירטסלקהיזכרמ
    """
    n_samples, n_features = X.shape
    centroids = np.zeros((k, n_features))

    # יארקאןשארזכרמרזב
    centroids[0] = X[np.random.randint(n_samples)]

    # מופסונםיזכרמרזב
    for j in range(1, k):
        # רתויבבורקהזכרמלהדוקנלכמעובירבםיקזרמבשח
        distances = np.array([
            min(np.linalg.norm(x - centroids[i])**2
                for i in range(j))
            for x in X
        ])

        # תורבתסהתסמטוגלפתהיפלשדחזכרמרזב
        probabilities = distances / distances.sum()
        cumulative_probs = probabilities.cumsum()
        r = np.random.rand()

        for i, prob in enumerate(cumulative_probs):
            if r < prob:
                centroids[j] = X[i]
                break

    return centroids

```

```

def kmeans(X, k, max_iters=100, tol=1e-4, init='kmeans++'):
    """
    K-Means++ לזכרון וזכרון K-Means++
    Parameters:
    X: array-like, shape (n_samples, n_features)
    k: int
    max_iters: int, default=100
    tol: float, default=1e-4
    init: str, default='kmeans++'
    Returns:
    centroids: array, shape (k, n_features)
    labels: array, shape (n_samples,)
    inertia: float
    (within-cluster sum of squares)
    """
    n_samples, n_features = X.shape

    # זכרון לזכרון
    if init == 'kmeans++':
        centroids = kmeans_plusplus_init(X, k)
    else:
        centroids = random_initialization(X, k)

    for iteration in range(max_iters):
        # שלב 1: (Assignment Step)
        # חלוקת הנקודות לזכרון
        distances = np.array([
            [np.linalg.norm(x - centroid)**2
             for centroid in centroids]
            for x in X
        ])
        labels = np.argmin(distances, axis=1)

        # שלב 2: (Update Step)
        # חלוקת הנקודות לזכרון
        new_centroids = np.array([
            X[labels == j].mean(axis=0)
            if np.any(labels == j)
            else centroids[j]
            for j in range(k)
        ])

```

טיפול בקלסטרים ריקים

```
def handle_empty_clusters(X, labels, centroids, k):
    """
    לוחטאטשודחמטידיטלעטסיקירטסירטסלקבלופיטטטטט
    טטטט"""
    new_centroids = centroids.copy()

    for j in range(k):
        if not np.any(labels == j): # קירטסלק
            # סירטסלקהיזכרמרתויבהקוחרההדוקנהתאצמ
            distances = np.min([
                np.linalg.norm(X - centroid, axis=1)
                for centroid in centroids
            ], axis=0)
            farthest_point_idx = np.argmax(distances)
            new_centroids[j] = X[farthest_point_idx]

    return new_centroids
```

5.8 שימוש ב-scikit-learn: הספרייה הסטנדרטית

בפועל, לצורכי ייצור (Production), מומלץ להשתמש ביישום המקצועי מספריית scikit-learn, שכולל אופטימיזציות רבות ומהיר משמעותית:

פרמטרים חשובים:

- `tini_n`: מספר הפעמים להריץ את האלגוריתם עם אתחולים שונים (ברירת מחדל: 10)

- `reti_xam`: מספר איטרציות מקסימלי

- `mhtirogla`: אלגוריתם לחישוב - 'dyoll' (קלאסי), 'nakle' (מהיר יותר), או 'otua'

5.9 סיכום והמלצות מעשיות

יישום מוצלח של K-Means דורש תשומת לב למספר היבטים קריטיים:

1. **אתחול:** השתמש ב-K-Means++ תמיד, אלא אם יש סיבה טובה לא לעשות זאת
2. **ריצות מרובות:** הרץ את האלגוריתם מספר פעמים ($n_{init} > 1$) ובחר את התוצאה הטובה ביותר
3. **קנה מידה:** נרמל את הנתונים (למשל, StandardScaler) כדי למנוע דומיננטיות של פיצ'רים עם טווח גדול

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# סייטתניססיןנותנרצי
X, y_true = make_blobs(n_samples=300, centers=4,
                        n_features=2, cluster_std=0.60,
                        random_state=42)

# תצורה K-Means
k = 4
centroids, labels, inertia = kmeans(X, k, init='kmeans++')

# היצזילאוזיו
plt.figure(figsize=(12, 5))

# סייתימאהסינותנה 1: קרג
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis',
            alpha=0.6, edgecolors='k')
plt.title('True Clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# סייתימאהסינותנה 2: קרג
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
            alpha=0.6, edgecolors='k')
plt.scatter(centroids[:, 0], centroids[:, 1],
            marker='X', s=300, c='red', edgecolors='black',
            linewidths=2, label='Centroids')
plt.title(f'K-Means Clustering (K={k})')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()

plt.tight_layout()
plt.savefig('images/kmeans_simulation.pdf', dpi=300,
            bbox_inches='tight')
print(f'Inertia (WCSS) : {inertia:.2f}')
```

```

from sklearn.cluster import KMeans

# לדומתריצי K-Means
kmeans_model = KMeans(n_clusters=4, init='k-means++',
                       n_init=10, max_iter=300,
                       random_state=42)

# סיננתנהלהצרה
kmeans_model.fit(X)

# תואצותלבק
labels = kmeans_model.labels_
centroids = kmeans_model.cluster_centers_
inertia = kmeans_model.inertia_

print(f'Number of iterations: {kmeans_model.n_iter_}')
print(f'Inertia: {inertia:.2f}')

```

4. **קריטריון עצירה:** בחר סף התכנסות (tol) סביר – לא קטן מדי (ייקח זמן רב) ולא גדול מדי (התכנסות מוקדמת)

5. **מורכבות חישובית:** זכור ש-K-Means הוא $O(n \cdot K \cdot d \cdot t)$ כאשר t הוא מספר האיטרציות

בפרק הבא נדון בשאלה הקריטית: כיצד בוחרים את K , מספר הקלסטרים?

6 מגבלות אלגוריתם K-Means: כאשר הפשטות הופכת למכשול

6.1 מחיר האלגנטיות

אלגוריתם K-Means, למרות יופיו המתמטי ויעילותו החישובית, סובל ממספר מגבלות משמעותיות המצמצמות את תחומי התחולה שלו. מגבלות אלו אינן תוצאה של יישום לקוי או אופטימיזציה בלתי-מושלמת, אלא נובעות מהנחות היסוד של האלגוריתם עצמו. כפי שאמר הסטטיסטיקאי הבריטי George E. P. Box: "כל המודלים שגויים, אך חלקם שימושיים" [11]. במקרה של K-Means, ההנחות המפשטות שהופכות אותו ליעיל הן בדיוק אלה שמגבילות אותו.

בפרק זה נבחן באופן ביקורתי את נקודות התורפה של האלגוריתם, נדגים מקרים שבהם הוא נכשל באופן דרמטי, ונבין את המחיר שאנו משלמים על הפשטות האלגוריתמית.

6.2 הנחת הקמירות: כאשר הגיאומטריה מכשילה

המגבלה המשמעותית ביותר של K-Means נובעת מפונקציית המטרה שלו - מזעור מרחקים אוקלידיים מנקודות למרכזים. פונקציה זו מובילה באופן טבעי לחלוקה לקלסטרים קמורים (Convex Clusters).

הגדרה: קבוצה $C \subseteq \mathbb{R}^d$ היא קמורה אם לכל שתי נקודות $x, y \in C$ והפרמטר $\lambda \in [0, 1]$, מתקיים:

$$(45) \quad \lambda x + (1 - \lambda)y \in C$$

כלומר, כל הקו המחבר בין כל שתי נקודות בקבוצה נמצא בתוך הקבוצה. **מסקנה:** K-Means מניח שהקלסטרים ה"אמיתיים" בנתונים הם קמורים. כאשר הנחה זו אינה מתקיימת, האלגוריתם נכשל.

6.3 דוגמה קלאסית: בעיית המעגלים המרוכזים

הדוגמה הקלאסית לכשלון K-Means היא מערך נתונים בצורת שני מעגלים קונצנטריים (מעגלים מרוכזים).

תיאור הבעיה:

- קלסטר 1: נקודות על מעגל פנימי ברדיוס $r_1 = 1$

- קלסטר 2: נקודות על מעגל חיצוני ברדיוס $r_2 = 3$

- שני הקלסטרים חולקים את אותו המרכז במקור

מתמטית, נוכל לתאר זאת:

$$C_1 = \{x \in \mathbb{R}^2 : \|x\| = r_1\} \quad (46)$$

$$C_2 = \{x \in \mathbb{R}^2 : \|x\| = r_2\} \quad (47)$$

מדוע K-Means נכשל?

עבור בעיה זו, מרכז המסה של כל אחד מהמעגלים הוא הראשית $(0,0)$. לכן, אם K-Means ימצב את שני המרכזים שלו בראשית (או קרוב לה), הוא לא יוכל להפריד בין המעגלים. במקום זאת, האלגוריתם יחלק את המישור לשני חצאים על ידי קו ישר העובר דרך הראשית - פתרון שאינו משקף כלל את המבנה האמיתי של הנתונים.

ניתוח גיאומטרי:

נניח ש-K-Means מכניס שני מרכזים μ_1 ו- μ_2 . גבול ההחלטה (Decision Boundary) בין הקלסטרים יהיה קבוצת הנקודות השווה-מרחק משני המרכזים:

$$(48) \quad \{x : \|x - \mu_1\| = \|x - \mu_2\|\}$$

זהו **היפר-מישור** (Hyperplane) - במקרה הדו-מימדי, קו ישר. לכן, גבולות ההחלטה של K-Means הם תמיד לינאריים!

מסקנה עמוקה: K-Means מניח שגבולות הקלסטרים הם לינאריים, ולכן לא יכול לזהות מבנים מורכבים כמו מעגלים, חצאי ירחים, או צורות לא-קמורות אחרות.

6.4 תלות באתחול: רגישות לתנאים התחלתיים

כפי שראינו בהוכחת ההתכנסות, K-Means מתכנס למינימום לוקלי, לא גלובלי. המשמעות: התוצאה הסופית תלויה באתחול.

דוגמה מספרית:

נחזור לדוגמה הפשוטה מפרק 11.3, אך נשנה את האתחול:

נתונים (זהים):

$$(49) \quad x_1 = (1, 1), \quad x_2 = (2, 1), \quad x_3 = (5, 5), \quad x_4 = (6, 5)$$

אתחול גרוע: נבחר שני מרכזים קרובים:

$$(50) \quad \mu_1^{(0)} = (1, 1), \quad \mu_2^{(0)} = (1.5, 1)$$

כעת, בשלב ההקצאה הראשון:

- x_1 יוקצה ל- μ_1 (מרחק 0)

- x_2 יוקצה ל- μ_2 (מרחק קטן)

- x_3, x_4 - יוקצו לאחד המרכזים באופן שרירותי

התוצאה: חלוקה לא-אופטימלית שונה לחלוטין מהחלוקה ה"טבעית".

פתרון מעשי: הרצת האלגוריתם מספר פעמים עם אתחולים שונים, ובחירת הפתרון עם

J המינימלי:

$$(51) \quad \text{Best clustering} = \arg \min_{i \in \{1, \dots, R\}} J_i$$

כאשר R הוא מספר ההרצות (בדרך כלל $R = 10$ עד $R = 100$).

6.5 בעיית בחירת K : כמה קלסטרים באמת?

K-Means דורש מהמשתמש לציין מראש את מספר הקלסטרים K . זוהי מגבלה משמעותית, שכן בלמידה לא-מפוקחת לרוב אין לנו מושג כמה קלסטרים קיימים בנתונים.
דילמה:

- אם K קטן מדי - נאבד מידע, נאחד קלסטרים שונים

- אם K גדול מדי - נפצל קלסטרים טבעיים, נוסיף רעש

שיטות לבחירת K :

שיטת המרפק (Elbow Method):

הרץ K-Means עבור $K = 1, 2, \dots, K_{\max}$ ושרטט את $J(K)$. חפש "מרפק" בגרף - נקודה שבה הירידה ב- J הופכת פחות משמעותית:

$$(52) \quad K^* = \arg \max_K \frac{J(K-1) - J(K)}{J(K) - J(K+1)}$$

קריטריון Silhouette: נדון בפרוטרוט בפרק 11.7.

קריטריון BIC/AIC: שיטות סטטיסטיות המאזנות בין איכות ההתאמה למורכבות המודל [12].

עם זאת, כל השיטות הללו הן היוריסטיות ואינן מבטיחות את K ה"נכון".

6.6 רגישות לחריגים

K-Means משתמש בממוצע חשבוני לחישוב מרכזי הקלסטרים. הממוצע רגיש מאוד **לחריגים** (Outliers) - נקודות קיצוניות רחוקות מרוב הנתונים.

משפט: יהי קלסטר C עם n נקודות, והוסף נקודת חריג $\mathbf{x}_{\text{out}}$ במרחק d גדול מהמרכז. המרכז החדש יזוז ב:

$$(53) \quad \Delta \mu = \frac{1}{n+1} (\mathbf{x}_{\text{out}} - \mu_{\text{old}})$$

כלומר, חריג אחד יכול להזיז את המרכז באופן משמעותי.

דוגמה: קלסטר עם 100 נקודות סביב $(0, 0)$, וחריג אחד ב- $(100, 100)$. המרכז החדש יהיה:

$$(54) \quad \mu_{\text{new}} = \frac{100 \cdot (0, 0) + 1 \cdot (100, 100)}{101} \approx (0.99, 0.99)$$

זהו מרחק משמעותי מהמרכז המקורי!

פתרון אפשרי: שימוש באלגוריתם K-Medoids (פרק 11.6) שמשתמש במדיאנה במקום ממוצע, והיא עמידה יותר לחריגים.

6.7 הנחת שווי גודל ושווי צפיפות

K-Means מניח בעקיפין שהקלסטרים דומים בגודל ובצפיפות. זאת מכיוון שפונקציית המטרה J מעניקה משקל שווה לכל נקודה.

בעיה: אם יש קלסטר גדול עם $n_1 = 1000$ נקודות וקלסטר קטן עם $n_2 = 10$ נקודות, K-Means עלול "לפצל" את הקלסטר הגדול כדי להקטין את J , תוך התעלמות מהקלסטר הקטן.

ניתוח מתמטי:

פונקציית המטרה היא:

$$(55) \quad J = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 = \sum_{k=1}^K |C_k| \cdot \text{Var}(C_k)$$

כאשר $\text{Var}(C_k)$ היא השונות הממוצעת של הקלסטר. לכן, קלסטר גדול עם שונות בינונית תורם יותר ל- J מקלסטר קטן עם שונות גדולה, והאלגוריתם "יעדיף" לפצל את הגדול.

6.8 קושי במרחבים רב-מימדיים

K-Means משתמש במרחק אוקלידי. כפי שראינו בפרק 11.1, במרחבים רב-מימדיים מושג "קרבה" הופך לא-מובחן בגלל קללת הממד:

$$(56) \quad \lim_{d \rightarrow \infty} \frac{\text{dist}_{\min}}{\text{dist}_{\max}} \rightarrow 1$$

השלכה: בממדים גבוהים ($d > 50$), כל הנקודות נראות "רחוקות" באותה מידה, ו-K-Means מאבד את יכולת ההפרדה שלו.

פתרון: הפחתת מימדים (PCA, t-SNE, UMAP) לפני יישום הקלסטרינג.

6.9 סיכום: מתי לא להשתמש ב-K-Means?

הימנע משימוש ב-K-Means כאשר:

- הקלסטרים אינם קמורים (מעגלים, צורות מורכבות)

- הקלסטרים בגדלים שונים משמעותית

- הקלסטרים בצפיפויות שונות

- יש הרבה חריגים בנתונים

- הנתונים במימדים גבוהים מאוד (עשרות ומעלה)

- אין לך מושג כמה קלסטרים צריכים להיות

- זקוק לגבולות החלטה לא-לינאריים

במקרים אלה, שקול אלגוריתמים חלופיים כמו:

- DBSCAN - מזהה צורות שרירותיות, עמיד לחריגים [13]

- Gaussian Mixture Models (GMM) - מאפשר קלסטרים בגדלים שונים

- Hierarchical Clustering - לא דורש K מראש

- Spectral Clustering - מטפל בצורות לא-קמורות

למרות מגבלות אלה, K-Means נשאר אלגוריתם שימושי ביותר כאשר הנחותיו מתקיימות - והוא משמש כנקודת התחלה מעולה לניתוח קלסטרינג ראשוני.

7 אלגוריתם K-Medoids: קלסטרינג עמיד בפני חריגים

7.1 מגבלת K-Means: רגישות לחריגים

אלגוריתם K-Means, למרות היעילות והפשטות שלו, סובל מחולשה מהותית: רגישות יתר לחריגים (Outliers). הסיבה לכך טמונה בשימוש במוצע לממונה מרכזי הקלסטרים. נזכור שבכל איטרציה, K-Means מחשב את מרכז הקלסטר כממוצע של כל הנקודות בקלסטר:

$$(57) \quad \mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$$

הבעיה: הממוצע רגיש מאוד לערכים קיצוניים. נקודה אחת חריגה עלולה "למשוך" את המרכז בכיוון שאינו מייצג את רוב הנקודות בקלסטר. זוהי תוצאה ישירה של העובדה ש-K-Means ממזער את סכום ריבועי המרחקים – חריגים רחוקים תורמים באופן דיספרופורציונלי לפונקציית המטרה.

דוגמה: נניח שיש לנו קלסטר של 100 נקודות מרוכזות סביב הנקודה (0,0), ונקודה חריגה אחת ב-(100,100). הממוצע יהיה קרוב ל-(1,1), רחוק מהמסה המרכזית של הנקודות.

7.2 הפתרון: Medoid במקום Centroid

K-Medoids הוא אלגוריתם קלסטרינג חלופי שפותר את בעיית הרגישות לחריגים. ההבדל המרכזי: במקום לחשב ממוצע (שעלול להיות נקודה "וירטואלית" שלא קיימת בנתונים), K-Medoids בוחר **נקודה אמיתית** מתוך הנתונים להיות מרכז הקלסטר.

הגדרה: Medoid הוא הנקודה במערך הנתונים שממזערת את סכום המרחקים (לא ריבועי המרחקים!) לכל הנקודות האחרות בקלסטר שלה:

$$(58) \quad \mathbf{m}_k = \arg \min_{\mathbf{x}_i \in C_k} \sum_{\mathbf{x}_j \in C_k} d(\mathbf{x}_i, \mathbf{x}_j)$$

כאשר $d(\cdot, \cdot)$ היא מטריקת מרחק כלשהי (לא בהכרח אוקלידית).
יתרונות:

1. **עמידות בפני חריגים:** שימוש בסכום מרחקים (לא ריבועים) והגבלה לנקודות קיימות מפחיתים משמעותית את ההשפעה של חריגים

2. **גמישות במטריקות:** ניתן להשתמש בכל מטריקת מרחק, לא רק אוקלידית

3. **פרשנות אינטואיטיבית:** ה-medoid הוא נקודה אמיתית מהנתונים, מה שמקל על פרשנות התוצאות

4. **התאמה לנתונים קטגוריאליים:** עובד עם נתונים בדידים ומעורבים

חסרונות:

1. מורכבות חישובית גבוהה: $O(n^2)$ בכל איטרציה (לעומת $O(n)$ ב-K-Means)

2. זמן ריצה ארוך: לא מתאים למערכי נתונים גדולים מאוד

7.3 אלגוריתם PAM: Partitioning Around Medoids

האלגוריתם הקלאסי ליישום K-Medoids הוא PAM (Partitioning Around Medoids), שפותח על ידי קאופמן ורוסו ב-1987 [14].

מבנה האלגוריתם:

PAM מורכב משני שלבים עיקריים:

שלב 1: BUILD Phase – בניית קבוצת Medoids ראשונית

בחר K נקודות כ-medoids ראשוניים באופן הבא:

1. בחר את הנקודה עם סכום המרחקים המינימלי לכל הנקודות האחרות כ-medoid ראשון

2. לכל $k = 2, 3, \dots, K$:

- עבור כל נקודה שעדיין לא נבחרה, חשב את ההפחתה בעלות הכוללת אם נבחר אותה כ-medoid

- בחר את הנקודה שמביאה להפחתה המקסימלית

שלב 2: SWAP Phase – שיפור איטרטיבי

חזור על התהליך הבא עד התכנסות:

1. עבור כל זוג (m_i, x_j) כאשר m_i הוא medoid ו- x_j אינו medoid:

- חשב את השינוי בעלות אם נחליף את m_i ב- x_j

-

$$(59) \quad \Delta_{ij} = \text{Cost}(\text{after swap}) - \text{Cost}(\text{before swap})$$

2. אם קיים זוג עם $\Delta_{ij} < 0$, בצע את ההחלפה שמביאה לשיפור המקסימלי

3. אם אין שיפור אפשרי, עצור

פונקציית העלות:

העלות הכוללת של חלוקה לקלסטרים:

$$(60) \quad \text{Cost} = \sum_{i=1}^n \min_{k=1, \dots, K} d(\mathbf{x}_i, \mathbf{m}_k)$$

כאשר $\mathbf{m}_k$ הם ה-medoids.

7.4 יישום PAM ב-Python

להלן מתווה כללי של יישום PAM. יישום מלא ומפורט של הקוד זמין בנספח ט' בסוף הפרק.
מבנה כללי של המחלקה:

מבנה כללי של MAP

```
class PAM:
    """Partitioning Around Medoids (PAM) Algorithm"""

    def __init__(self, n_clusters=3, max_iter=100):
        # סירטמרפלוחתא
        pass

    def fit(self, X, distance_matrix=None):
        # סיקחרמתצירטמבושיח: 1 בלש
        # 2: BUILD - לוחתא medoids
        # 3: SWAP - ביטרטיארופיש
        # 4: סירטסלקליפוסדויש
        pass

    def _build_phase(self, D):
        # סיינושאר K medoids תריחב
        pass

    def _swap_phase(self, D):
        # לשיביטרטיארופיש medoids
        pass

    def _compute_swap_cost(self, D, medoid_pos, candidate):
        # תפלחהתולעבושיח medoid
        pass
```

רכיבי המימוש המרכזיים:

1. חישוב מטריצת מרחקים: מטריצה סימטרית D_{ij} בגודל $n \times n$ שבה כל איבר מייצג מרחק בין שתי נקודות
2. שלב DLIUB: בחירה תחכמנית של K medoids ראשוניים המפזרת אותם במרחב
3. שלב PAWS: איטרציות חוזרות של בדיקת החלפות והחלפה של ה-medoid המשפר ביותר
4. קריטריון עצירה: כאשר אין החלפה שמשפרת את העלות הכוללת

עלות חישובית של כל פעולה:

- חישוב מטריצת מרחקים: $O(n^2d)$

- שלב DLIUB: $O(n^2K)$

- איטרציה של PAWS: $O(n^2K)$

- סה"כ: $O(n^2K \cdot T)$ כאשר T הוא מספר האיטרציות

להמשך מפורט, ראו נספח ט' בסוף הפרק ליישום המלא עם הסברים מפורטים על כל שלב.

7.5 דוגמה מעשית: קלסטרינג מדינות

נדגים את היתרון של K-Medoids בעזרת דוגמה מעשית: קלסטרינג מדינות לפי מרחק גיאוגרפי. במקרה זה, medoid מייצג מדינה "מרכזית" בכל קלסטר.

7.6 שימוש ב-scikit-learn-extra

בפועל, קיים יישום מקצועי של K-Medoids בספרייה artxe-nrael-tikics:
פרמטרים חשובים:

- dohtem: 'map' (מדויק אך איטי) או 'etanretla' (מהיר יותר אך פחות מדויק)

- tini: 'modnar', 'citsirueh', או '++sdiodem-k' (מומלץ)

- cirtem: מטריקת מרחק - 'naedilcue', 'nattahnam', 'enisoc', וכו'

7.7 השוואה: K-Means לעומת K-Medoids

וויזטירק	K-Means	K-Medoids
רטסלק זכרמ	(ילאוטריו) עצוממ	(medoid) תיתימא הדוקנ
סיגירחל תושיגר	דואמ ההובג	הכומנ
קחרמ תקירטמ	תידילקוא קר	הקירטמ לכ
היצרטיאל תובכרומ	$O(nKd)$	$O(n^2K)$
הציר נמז	ריהמ	יטיא
סילודג סינותנל המאתה	ניוצמ	עורג
סיילאירוגטק סינותנ	אל	(המיאתמ הקירטמ סע) נכ
תונשרפ	השק	הלק

7.8 מתי להשתמש ב-K-Medoids?

השתמש ב-K-Medoids כאשר:

1. יש חריגים במערך הנתונים ואתה מעוניין באלגוריתם עמיד

```
import numpy as np
import matplotlib.pyplot as plt

# תוניסיה (latitude, longitude)
countries = {
    'Israel': (31.7683, 35.2137),
    'Egypt': (26.8206, 30.8025),
    'Jordan': (31.9454, 35.9284),
    'Lebanon': (33.8547, 35.8623),
    'Syria': (34.8021, 38.9968),
    'Turkey': (38.9637, 35.2433),
    'Greece': (39.0742, 21.8243),
    'Italy': (41.8719, 12.5674),
    'Spain': (40.4637, -3.7492),
    'France': (46.2276, 2.2137),
    'Germany': (51.1657, 10.4515),
    'Poland': (51.9194, 19.1451),
    'Ukraine': (48.3794, 31.1656),
    'Russia': (61.5240, 105.3188)
}

# דריסת יחידות התרחות numpy
country_names = list(countries.keys())
X = np.array(list(countries.values()))

# תצורה PAM
n_clusters = 4
pam = PAM(n_clusters=n_clusters, max_iter=100)
pam.fit(X)

# תספדה medoids
print("Medoids (תוניסיה):")
for i, medoid_idx in enumerate(pam.medoid_indices_):
    print(f"Cluster {i+1}: {country_names[medoid_idx]}")

# היצור לאוזיו
plt.figure(figsize=(12, 8))
colors = ['red', 'blue', 'green', 'orange']

for i, country in enumerate(country_names):
    cluster = pam.labels_[i]
    marker = 'X' if i in pam.medoid_indices_ else 'o'
    size = 300 if i in pam.medoid_indices_ else 100

    plt.scatter(X[i, 1], X[i, 0],
                c=colors[cluster], marker=marker, s=size,
                edgecolor='black', linewidths=2,
                alpha=0.7)

    plt.annotate(country, (X[i, 1], X[i, 0]),
```



```

from sklearn_extra.cluster import KMedoids

# לדוגמה K-Medoids
kmedoids = KMedoids(n_clusters=4, method='pam',
                    init='k-medoids++', max_iter=100,
                    random_state=42)

# סינון והלעה
kmedoids.fit(X)

# תוצאות
labels = kmedoids.labels_
medoid_indices = kmedoids.medoid_indices_
inertia = kmedoids.inertia_

print(f"Medoid countries:")
for idx in medoid_indices:
    print(f"_{country_names[idx]}")

print(f"\nInertia:_{inertia:.2f}")

```

2. מטריקת המרחק אינה אוקלידית (למשל, מרחק קוסינוס, מרחק Manhattan)

3. הנתונים אינם מספריים (למשל, סטרינגים, קטגוריות)

4. חשוב שמרכזי הקלסטרים יהיו נקודות אמיתיות לצורכי פרשנות

5. מערך הנתונים קטן-בינוני ($n < 10,000$)

השתמש ב-K-Means כאשר:

1. מערך הנתונים גדול ($n > 100,000$)

2. אין חריגים משמעותיים

3. מרחק אוקלידי הגיוני למערך הנתונים

4. מהירות היא קריטית

7.9 וריאציות של K-Medoids

1. Clustering Large Applications – CLARA

פותח לטיפול במערכי נתונים גדולים. הרעיון: הרץ PAM על דגימות אקראיות מהנתונים, ובחר את הפתרון הטוב ביותר.

2. Clustering Large Applications based on RANdomized Search – CLARANS

שיפור של CLARA המשתמש בחיפוש אקראי במרחב החיפוש של medoids אפשריים.

3. FastPAM

אלגוריתם משופר של PAM עם מורכבות $O(n \cdot K)$ במקום $O(n^2)$, שפותח על ידי שוברט ורוסו ב-2019 [15].

7.10 סיכום: K-Medoids כחלופה עמידה

K-Medoids הוא אלגוריתם קלסטרינג חזק המספק פתרון אלגנטי לבעיית הרגישות לחריגים של K-Means. המחיר הוא מורכבות חישובית גבוהה יותר, אך עבור מערכי נתונים בגדלים בינוניים, זהו מחיר ששווה לשלם.

ההבנה שקלסטרינג אינו "פתרון יחיד שמתאים לכולם" היא קריטית. בחירת האלגוריתם הנכון תלויה באופי הנתונים, גודל מערך הנתונים, ומטריקת המרחק הטבעית לבעיה. K-Medoids מציע כלי נוסף בארגז הכלים של מדען הנתונים, ובמצבים המתאימים, הוא עשוי להיות הבחירה המועדפת.

8 ניתוח Silhouette: מדידת איכות הקלסטרינג

8.1 השאלה המרכזית: האם הקלסטרינג טוב?

לאחר שהרצנו אלגוריתם קלסטרינג כלשהו - K-Means, K-Medoids, או כל אלגוריתם אחר - עומדת בפנינו שאלה קריטית: עד כמה הקלסטרינג שקיבלנו הוא טוב? שאלה זו מורכבת במיוחד בלמידה לא-מפוקחת, שכן אין לנו תוויות "אמת" להשוות אליהן. מדד ה-Silhouette, שפותח על ידי Peter J. Rousseeuw בשנת 1987 [16], מציע פתרון אלגנטי לבעיה זו. המדד מודד עבור כל נקודה עד כמה היא "מתאימה" לקלסטר שלה לעומת הקלסטרים האחרים, ומספק הערכה כמותית לאיכות הקלסטרינג הכוללת. הרעיון מבריק בפשטותו: נקודה "שייכת" באמת לקלסטר שלה אם היא קרובה לנקודות בקלסטר שלה ורחוקה מנקודות בקלסטרים אחרים. זהו בדיוק מה שמדד ה-Silhouette מכמת.

8.2 הגדרה מתמטית: מדד ה-Silhouette

עבור נקודת נתונים x_i המשויכת לקלסטר C_k , נגדיר שני מדדים:

(1) **הלכידות הפנימית (Cohesion) - $a(i)$** :

ממוצע המרחקים מ- x_i לכל שאר הנקודות באותו קלסטר:

$$(61) \quad a(i) = \frac{1}{|C_k| - 1} \sum_{j \in C_k, j \neq i} d(x_i, x_j)$$

כאשר $d(x_i, x_j)$ הוא מרחק (לרוב אוקלידי) בין הנקודות. ככל ש- $a(i)$ קטן יותר, הנקודה קרובה יותר לשכנותיה בקלסטר - זה טוב!

(2) **ההפרדה החיצונית (Separation) - $b(i)$** :

המרחק הממוצע המינימלי לקלסטר אחר הקרוב ביותר. עבור כל קלסטר C_l (כאשר $l \neq k$), נחשב:

$$(62) \quad b(i) = \min_{l \neq k} \left\{ \frac{1}{|C_l|} \sum_{j \in C_l} d(x_i, x_j) \right\}$$

כלומר, $b(i)$ הוא המרחק הממוצע של x_i לקלסטר הזר הקרוב ביותר. ככל ש- $b(i)$ גדול יותר, הנקודה רחוקה מקלסטרים אחרים - זה טוב!

מקדם Silhouette לנקודה i :

כעת נשלב את שני המדדים למקדם יחיד:

$$(63) \quad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

נשים לב למספר נקודות מפתח:

- אם $a(i) \ll b(i)$ (נקודה קרובה לקלסטר שלה, רחוקה מאחרים): $s(i) \approx 1$ - מצוין!

- אם $a(i) \approx b(i)$ (נקודה באמצע בין קלסטרים): $s(i) \approx 0$ - לא ברור לאיזה קלסטר היא שייכת

- אם $a(i) \gg b(i)$ (נקודה קרובה יותר לקלסטר אחר): $s(i) \approx -1$ - הקצאה שגויה!
טווח ערכים: $s(i) \in [-1, 1]$.

8.3 פרשנות המקדם: מה אומר כל ערך?

נפרש את משמעות הערכים השונים של $s(i)$:
 $s(i) \approx 1$ (בין 0.7 ל-1.0): הנקודה משויכת בצורה מצוינת לקלסטר שלה. היא קרובה מאוד לנקודות בקלסטר, ורחוקה מקלסטרים אחרים. זוהי הקצאה חזקה ובטוחה.
 $s(i) \approx 0$ (בין -0.1 ל-0.1): הנקודה נמצאת על הגבול בין שני קלסטרים. לא ברור באופן חד-משמעי לאיזה קלסטר היא שייכת. זוהי הקצאה חלשה.
 $s(i) < 0$ (שלילי): הנקודה הוקצתה לקלסטר הלא-נכון! היא קרובה יותר לקלסטר אחר מאשר לקלסטר הנוכחי שלה. זוהי הקצאה שגויה.

מקדם Silhouette ממוצע:

עבור כל הנתונים, המקדם הממוצע הוא:

$$\bar{s} = \frac{1}{n} \sum_{i=1}^n s(i) \quad (64)$$

ערך $\bar{s}$ גבוה מעיד על קלסטרינג טוב. כלל אצבע:

- $\bar{s} > 0.7$ - מבנה קלסטרינג חזק ומובהק

- $0.5 < \bar{s} < 0.7$ - מבנה קלסטרינג סביר

- $0.25 < \bar{s} < 0.5$ - מבנה קלסטרינג חלש

- $\bar{s} < 0.25$ - אין מבנה קלסטרינג משמעותי

8.4 שימוש במדד לבחירת K האופטימלי

אחד היישומים החשובים ביותר של מדד ה-Silhouette הוא בחירת מספר הקלסטרים K . האלגוריתם:

- הרץ קלסטרינג עבור $K = 2, 3, \dots, K_{\max}$

- עבור כל K , חשב את $\bar{s}(K)$ - המקדם הממוצע

- בחר את ה- K שממקסם את $\bar{s}$:

$$(65) \quad K^* = \arg \max_K \bar{s}(K)$$

למה זה עובד?

כאשר K קטן מדי, קלסטרים שונים מאוחדים יחד, והמרחק הפנימי $a(i)$ גדל - המקדם $s(i)$ קטן. כאשר K גדול מדי, נקודות מאותו קלסטר טבעי מתפצלות, והמרחק לקלסטר הקרוב ביותר $b(i)$ קטן - שוב $s(i)$ קטן. רק עבור K "הנכון", שני המדדים מאוזנים והמקדם ממוקסם.

יתרון על שיטת המרפק:

בניגוד לשיטת המרפק (פרק 11.5) שמחפשת שינוי בשיפוע, מדד ה-Silhouette נותן ערך מוחלט - קל יותר לפרש ולהחליט.

8.5 דוגמה מספרית מפורטת

נדגים את החישוב על מערך נתונים פשוט עם $K = 2$ קלסטרים:
נתונים:

$$C_1 = \{(1, 1), (2, 1), (1.5, 1.5)\} \quad (66)$$

$$C_2 = \{(5, 5), (6, 5), (5.5, 5.5)\} \quad (67)$$

נחשב את $s(i)$ עבור הנקודה $\mathbf{x}_1 = (1, 1)$ מקלסטר C_1 .

שלב 1 - חישוב $a(1)$:

המרחק הממוצע לנקודות אחרות ב- C_1 :

$$d(\mathbf{x}_1, \mathbf{x}_2) = \|(1, 1) - (2, 1)\| = 1 \quad (68)$$

$$d(\mathbf{x}_1, \mathbf{x}_3) = \|(1, 1) - (1.5, 1.5)\| = \sqrt{0.5} \approx 0.71 \quad (69)$$

לכן:

$$(70) \quad a(1) = \frac{1 + 0.71}{2} = 0.855$$

שלב 2 - חישוב $b(1)$:

המרחק הממוצע לנקודות ב- C_2 :

$$d(\mathbf{x}_1, (5, 5)) = \sqrt{(1-5)^2 + (1-5)^2} = \sqrt{32} \approx 5.66 \quad (71)$$

$$d(\mathbf{x}_1, (6, 5)) = \sqrt{(1-6)^2 + (1-5)^2} = \sqrt{41} \approx 6.40 \quad (72)$$

$$d(\mathbf{x}_1, (5.5, 5.5)) = \sqrt{(1-5.5)^2 + (1-5.5)^2} = \sqrt{40.5} \approx 6.36 \quad (73)$$

לכן:

$$(74) \quad b(1) = \frac{5.66 + 6.40 + 6.36}{3} = 6.14$$

שלב 3 - חישוב $s(1)$:

$$(75) \quad s(1) = \frac{b(1) - a(1)}{\max\{a(1), b(1)\}} = \frac{6.14 - 0.855}{6.14} \approx 0.86$$

פרשנות: $s(1) = 0.86$ הוא ערך גבוה מאוד, המעיד שהנקודה $(1, 1)$ משויכת בצורה מצוינת לקלסטר C_1 !
באופן דומה נחשב את $s(i)$ עבור כל הנקודות, ונקבל את $\bar{s}$ הממוצע.

8.6 מורכבות חישובית

חישוב מקדם Silhouette לכל הנקודות דורש:
עבור כל נקודה i :

- חישוב $a(i)$: $O(|C_k|)$ פעולות

- חישוב $b(i)$: $O(n)$ פעולות (צריך לבדוק את כל הקלסטרים)

סה"כ עבור כל הנתונים: $O(n^2)$.

זוהי מורכבות נמוכה יחסית, והמדד יכול להיות מחושב ביעילות גם עבור מערכי נתונים גדולים (אלפי נקודות).

8.7 ויזואליזציה: תרשים Silhouette

דרך נפוצה להצגת מדד ה-Silhouette היא **תרשים Silhouette** (Silhouette Plot):
מבנה התרשים:

- ציר האופקי: ערכי $s(i)$ (מ-1 ל-1)

- ציר האנכי: נקודות הנתונים, מקובצות לפי קלסטר

- כל קלסטר מיוצג בצבע שונה

- עבור כל נקודה, מצוירת פס אופקי באורך $s(i)$

פרשנות:

- פסים ארוכים לכיוון הימין (חיובי) - הקצאות טובות

- פסים קצרים או בכיוון השמאלי (שלילי) - הקצאות בעייתיות

- קלסטרים בגדלים דומים - מבנה מאוזן

- קו אנכי ב- $\bar{s}$ - המקדם הממוצע

8.8 יתרונו וחסרונות של מדד Silhouette

יתרונות:

- פשטות קונספטואלית: הרעיון אינטואיטיבי - קרבה למרכז, ריחוק מאחרים
 - ערך מנורמל: תמיד ב- $[-1, 1]$, קל לפרש
 - ללא תלות במודל: עובד עם כל אלגוריתם קלסטרינג
 - זיהוי בעיות: מראה אילו נקודות הוקצו לא-נכון
 - בחירת K : כלי מעשי לבחירת מספר הקלסטרים
- חסרונות:

- הנחת קמירות: מניח קלסטרים קמורים וצפופים - לא עובד טוב עם צורות מורכבות
- רגישות למרחק: תלוי במטריקת המרחק שנבחרה
- מורכבות $O(n^2)$: לא אידיאלי למערכי נתונים ענקיים (מיליוני נקודות)
- לא מדד מושלם: מקדם גבוה לא מבטיח שהקלסטרינג "נכון" במובן סמנטי

8.9 השוואה לשיטות אחרות

Silhouette מול Elbow Method:

- שיטת המרפק (פרק 11.5) מסתמכת על פונקציית המטרה J של האלגוריתם עצמו, בעוד ש-Silhouette הוא מדד חיצוני עצמאי. לכן Silhouette נותן הערכה אובייקטיבית יותר.
- Silhouette מול מדדים פנימיים אחרים:
- קיימים מדדים נוספים כמו Davies-Bouldin Index [17] ו-Calinski-Harabasz Index [18]. Silhouette מתייחד בכך שהוא מספק מידע ברמת הנקודה הבודדת, לא רק בממוצע כולל.

8.10 סיכום: מתי להשתמש ב-Silhouette?

השתמש במדד Silhouette:

- לבחירת מספר הקלסטרים K האופטימלי
 - להערכת איכות קלסטרינג שהתקבל
 - לזיהוי נקודות שהוקצו לא-נכון (נקודות עם $s(i) < 0$)
 - להשוואה בין אלגוריתמי קלסטרינג שונים על אותם נתונים
 - כאשר הקלסטרים הצפויים הם קמורים וצפופים
- מדד ה-Silhouette, למרות פשטותו, מספק כלי עוצמתי להערכת איכות קלסטרינג והפך לאחד המדדים הנפוצים ביותר בתחום. הוא ממחיש יפה כיצד מושגים מתמטיים פשוטים - קרבה פנימית וריחוק חיצוני - יכולים להוביל לכלי פרקטי ושימושי.

9 הקשר ללמידת מכונה מודרנית: K-Means בעידן הרשתות העמוקות

9.1 מהיכן באנו ולכן אנחנו הולכים

אלגוריתם K-Means, שפותח לראשונה על ידי סטיוארט לוי ב-1957 [7], עשוי להיראות כשריד ארכאולוגי בעידן של Transformers, GPT, ומודלים עם מיליארדי פרמטרים. אולם, דווקא בעידן זה, כאשר מודלים מתמודדים עם מרחבי ייצוג (Representation Spaces) בני אלפי מימדים, K-Means מוצא לעצמו מקום חדש ומרתק.

הפרק הזה בוחן כיצד אלגוריתם קלסטרינג פשוט מהמאה ה-20 הפך לכלי חיוני בארגון הכלים של למידת מכונה מודרנית. נחקור את התפקיד שלו באימון רשתות נוירונים, בלמידה בייצוג (Representation Learning), ובארכיטקטורות מתקדמות כמו Vector Quantized Variational Autoencoders.

9.2 K-Means כשלב קדם-עיבוד: Feature Learning

אחת השימושים המודרניים החשובים של K-Means היא כשיטה ללמידת פיצ'רים (Feature Learning). הרעיון המרכזי: השתמש ב-K-Means כדי ללמוד מילון של תבניות בסיסיות במערכת הנתונים, ואז ייצג כל דוגמה כשילוב של תבניות אלו.

האלגוריתם:

1. הרץ K-Means על מערך הנתונים לקבלת K מרכזי קלסטרים $\{\mu_k\}_{k=1}^K$

2. עבור כל דוגמה $\mathbf{x}_i$, צור ייצוג חדש:

$$(76) \quad \mathbf{f}(\mathbf{x}_i) = [d(\mathbf{x}_i, \mu_1), d(\mathbf{x}_i, \mu_2), \dots, d(\mathbf{x}_i, \mu_K)]$$

כאשר $d(\cdot, \cdot)$ היא מטריקת מרחק

3. השתמש ב- $\mathbf{f}(\mathbf{x}_i)$ כפיצ'רים עבור מודל למידה מפוקחת

וריאציה: קידוד קשיח (Hard Coding)

במקום להשתמש במרחקים, השתמש בקידוד one-hot של הקלסטר הקרוב ביותר:

$$(77) \quad f_k(\mathbf{x}_i) = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_i - \mu_j\| \\ 0 & \text{otherwise} \end{cases}$$

וריאציה: קידוד רך (Soft Coding)

השתמש בהסתברויות על בסיס מרחקים:

$$(78) \quad f_k(\mathbf{x}_i) = \frac{\exp(-\beta \|\mathbf{x}_i - \mu_k\|^2)}{\sum_{j=1}^K \exp(-\beta \|\mathbf{x}_i - \mu_j\|^2)}$$

כאשר $\beta > 0$ הוא פרמטר "טמפרטורה" השולט ברמת ה"רכות".

9.3 K-Means ורשתות נוירונים: קשרים מפתיעים

קיים קשר עמוק ומפתיע בין K-Means לבין רשתות נוירונים, במיוחד רשתות מסוג Radial Basis Function (RBF).

רשת RBF כהכללה של K-Means:

רשת RBF היא רשת נוירונים בעלת שתי שכבות:

1. שכבת RBF: K נוירונים, כל אחד מייצג "מרכז" μ_k ומחשב:

$$(79) \quad \phi_k(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mu_k\|^2}{2\sigma^2}\right)$$

2. שכבה ליניארית: שכבת פלט המשקללת את פלטי ה-RBF:

$$(80) \quad f(\mathbf{x}) = \sum_{k=1}^K w_k \phi_k(\mathbf{x})$$

הקשר ל-K-Means:

אם נגדיר $\sigma \rightarrow 0$, פונקציות ה-RBF הופכות לפונקציות אינדיקטור:

$$(81) \quad \lim_{\sigma \rightarrow 0} \phi_k(\mathbf{x}) = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x} - \mu_j\| \\ 0 & \text{otherwise} \end{cases}$$

במילים אחרות: K-Means הוא מקרה פרטי של רשת RBF כאשר $\sigma \rightarrow 0$.

9.4 Vector Quantization ו-VQ-VAE

אחד השימושים המתקדמים ביותר של K-Means בלמידה עמוקה הוא בכימות וקטורי (Vector Quantization, VQ), במיוחד בארכיטקטורת VQ-VAE שפותחה על ידי ואן דן אורד ועמיתיו ב-2017 [19].

הרעיון המרכזי:

במקום לקודד תמונה למרחב רציף (Continuous Latent Space) כמו ב-VAE רגיל, קודד אותה למרחב בזיד של K וקטורי קוד (Codebook Vectors).

ארכיטקטורת VQ-VAE:

1. Encoder: מעביר את התמונה $\mathbf{x}$ לייצוג רציף $\mathbf{z}_e(\mathbf{x})$

2. כימות וקטורי: מוצא את וקטור הקוד הקרוב ביותר:

$$(82) \quad \mathbf{z}_q(\mathbf{x}) = \arg \min_{\mathbf{e}_k \in \mathcal{C}} \|\mathbf{z}_e(\mathbf{x}) - \mathbf{e}_k\|$$

```

import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

def kmeans_feature_learning(X_train, X_test, n_clusters=100,
                             encoding='soft', beta=1.0):
    """
    K-Means תועצמאב'ס ירציפ תדימל
    Parameters:
    -----
    X_train, X_test: array-like
        הקידבול ומיא'נות
    n_clusters: int
        (וליהם) לדוגם ירטסלקה רפסמ
    encoding: str
        'hard', 'soft', 'distance'
    beta: float
        soft encoding רובע הרוטרפ מטרפ
    """
    # ומיאהינותנלע K-Means ןמא
    kmeans = KMeans(n_clusters=n_clusters, n_init=10,
                    random_state=42)
    kmeans.fit(X_train)

    def encode(X, method='soft'):
        """שדח'ס ירציפ תודוקנ תודוקנ"""
        distances = kmeans.transform(X) # מיזכרמלסיקחרמ

        if method == 'hard':
            # One-hot encoding
            labels = kmeans.predict(X)
            encoded = np.zeros((X.shape[0], n_clusters))
            encoded[np.arange(X.shape[0]), labels] = 1

        elif method == 'soft':
            # Soft assignment based on distances
            exp_dists = np.exp(-beta * distances**2)
            encoded = exp_dists / exp_dists.sum(axis=1, keepdims=True)

        else: # distance
            # Use distances directly
            encoded = distances

    return encoded

```

כאשר $\mathcal{C} = \{\mathbf{e}_1, \dots, \mathbf{e}_K\}$ הוא ספר הקודים (Codebook)

3. **Decoder**: משחזר את התמונה מ- $\mathbf{z}_q(\mathbf{x})$

זה בדיוק K-Means!

שלב הכימות הוקטורי זהה לשלב השיוך ב-K-Means. ההבדל: ספר הקודים נלמד במקביל לאנקודר והדקודר באמצעות backpropagation.
פונקציית האובדן:

$$(83) \quad \mathcal{L} = \underbrace{\|\mathbf{x} - \text{Decoder}(\mathbf{z}_q)\|^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[\mathbf{z}_e] - \mathbf{e}\|^2}_{\text{codebook}} + \underbrace{\beta \|\mathbf{z}_e - \text{sg}[\mathbf{e}]\|^2}_{\text{commitment}}$$

כאשר sg הוא tneidarg-pots (עצירת הגרדיאנט).

9.5 K-Means ב-Self-Supervised Learning

בשנים האחרונות, K-Means מצא שימוש חדש ומרתק בלמידה עצמית-מפוקחת (Self-Supervised Learning). אלגוריתמים כמו **DeepCluster** ו-**SwAV** משתמשים בקלסטרינג כדי ליצור "פסאודו-תוויות" (Pseudo-labels) לאימון רשתות נוירונים.
אלגוריתם DeepCluster (la te noraC, 2018): [20]

1. אמן רשת נוירונים לחילוץ פיצ'רים מתמונות

2. הרץ K-Means על הפיצ'רים המופקים

3. השתמש בשיוכי הקלסטרים כתוויות לאימון מפוקח

4. חזור על התהליך

פורמלית, ממזערים:

$$(84) \quad \min_{\theta, C} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

כאשר f_{θ} היא הרשת הנוירונית, $y_i = \arg \min_k \|\text{features}(\mathbf{x}_i) - \mu_k\|$ הוא שיוך הקלסטר, ו- ℓ היא פונקציית אובדן סיווג.

למה זה עובד?

הרעיון הוא שהרשת לומדת ייצוגים שמושכים תמונות דומות קרוב זו לזו במרחב הפיצ'רים. K-Means "מכריח" את הרשת ליצור מבנה מקובץ, מה שמוביל ללמידת ייצוגים משמעותיים.

9.6 K-Means לאתחול רשתות נוירונים

שימוש מעניין נוסף: אתחול משקלי רשתות נוירונים באמצעות K-Means.
השיטה:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class VectorQuantizer(nn.Module):
    """
    Vector Quantization Layer - VQ-VAE
    """

    def __init__(self, num_embeddings, embedding_dim, commitment_cost):
        """
        Parameters:
        -----
        num_embeddings: int
        מידוקה ירוטקו הרפסמ (K)
        embedding_dim: int
        רוטקו גלכדמי
        commitment_cost: float
        מתובי יחתה גלדבוא לקשמ (beta)
        """
        super().__init__()

        self.embedding_dim = embedding_dim
        self.num_embeddings = num_embeddings
        self.commitment_cost = commitment_cost

        # מידוקה רפס (learnable)
        self.embedding = nn.Embedding(num_embeddings, embedding_dim)
        self.embedding.weight.data.uniform_(
            -1.0 / num_embeddings, 1.0 / num_embeddings
        )

    def forward(self, inputs):
        """
        Parameters:
        -----
        inputs: tensor, shape (batch, embedding_dim, H, W)
        רדוקנאה טלפ

        Returns:
        -----
        quantized: tensor
        טנווקמה גוצייה
        loss: tensor
        VQ-ה גלדבוא
        """
        # Flatten input
        input_shape = inputs.shape
        flat_input = inputs.view(-1, self.embedding_dim)

```

1. דגום מספר גדול של חלונות (patches) קטנים מהתמונות
 2. הרץ K-Means על החלונות
 3. השתמש במרכזי הקלסטרים כמסננים (filters) בשכבה הראשונה של רשת קונבולוציונית
- קוואן ועמיתיו ב-2012 [21] הראו שאתחול זה יכול להוביל לביצועים טובים יותר, במיוחד כאשר יש מעט נתונים מתוייגים.

9.7 K-Means ב-Embedding Spaces

בעולם המודלים השפתיים הגדולים (LLMs) ומערכות Retrieval-Augmented Generation (RAG), משמש K-Means לארגון מרחבי embedding עצומים. שימושים טיפוסיים:

1. חיפוש יעיל: קלסטור embeddings של מסמכים מאפשר חיפוש היררכי מהיר
2. זיהוי נושאים: קלסטרים מייצגים נושאים סמנטיים שונים
3. דחיסה: ייצוג כל מסמך על ידי הקלסטר שלו חוסך זיכרון
4. מגוון תוצאות: בחר תוצאות מקלסטרים שונים להבטחת מגוון

9.8 אתגרים ופתרונות מודרניים

1. קנה מידה (Scalability): עבור מערכי נתונים ענקיים, K-Means הקלאסי איטי מדי. פתרונות:
 - Mini-Batch K-Means: עובד על תתי-מדגמים קטנים בכל איטרציה
 - Approximate K-Means: שימוש ב-Approximate Nearest Neighbors (ANN) לחיפוש מרכזים
 - חישוב מבוזר: שימוש ב-Spark או Dask למערכי נתונים מסיביים
2. מימדיות גבוהה: במרחבי embedding בני אלפי מימדים, מרחק אוקלידי הופך לפחות משמעותי. פתרונות:
 - הפחתת מימדים: PCA, UMAP, או t-SNE לפני קלסטרינג
 - מטריקות חלופיות: קוסינוס, מהלנוביס
 - קלסטרינג בתת-מרחבים: Subspace Clustering

```

import torch
import torch.nn as nn
from sklearn.cluster import KMeans
import numpy as np

def initialize_conv_with_kmeans(images, n_filters=64,
                                patch_size=5, n_patches=100000):
    """
    K-Means clustering for initializing convolutional filters.
    Parameters:
    images: array of shape (n_images, channels, height, width)
    patches: list to store extracted patches
    n_filters: number of filters (K)
    patch_size: size of the patch
    n_patches: number of patches to extract
    """
    channels = images.shape[1]
    patches = []

    # Extract patches
    for _ in range(n_patches):
        # Random image index
        img_idx = np.random.randint(0, len(images))
        img = images[img_idx]

        # Random patch coordinates
        h = np.random.randint(0, img.shape[1] - patch_size)
        w = np.random.randint(0, img.shape[2] - patch_size)

        # Extract patch
        patch = img[:, h:h+patch_size, w:w+patch_size]
        patches.append(patch.flatten())

    patches = np.array(patches)

    # Normalize
    patches = (patches - patches.mean(axis=0)) / (patches.std(axis=0) + 1e-10)

    # K-Means
    kmeans = KMeans(n_clusters=n_filters, n_init=10, random_state=42)
    kmeans.fit(patches)

```

```

from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
import numpy as np

# לדוגמה embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')

# סיכומות מסמכים
documents = [
    "Machine learning is a subset of artificial intelligence.",
    "Deep learning uses neural networks with many layers.",
    "Python is a popular programming language.",
    "JavaScript is used for web development.",
    "K-Means is a clustering algorithm.",
    "Support vector machines are used for classification.",
    # ... סיכומות נוספות
]

# embeddings
embeddings = model.encode(documents)

# קוטט לק
n_clusters = 3
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
clusters = kmeans.fit_predict(embeddings)

# תוצאות
for cluster_id in range(n_clusters):
    cluster_docs = [doc for doc, c in zip(documents, clusters)
                     if c == cluster_id]
    print(f"\nCluster {cluster_id}:")
    for doc in cluster_docs:
        print(f"  {doc}")

# וכתבשפחזא , יטנוולררטטלקאצמסדוק : יכריהשופיח
query = "What is deep learning?"
query_embedding = model.encode([query])[0]

# בורקרטטלקאצמ
cluster_distances = [
    np.linalg.norm(query_embedding - centroid)
    for centroid in kmeans.cluster_centers_
]
closest_cluster = np.argmin(cluster_distances)

print(f"\nQuery: {query}")
print(f"Relevant cluster: {closest_cluster}")

```

9.9 סיכום: K-Means בני אלמוות

אלגוריתם K-Means, למרות פשטותו, ממשיך להיות כלי חיוני בלמידת מכונה מודרנית. הוא מצא שימושים חדשים בלמידה עמוקה, למידה עצמית-מפוקחת, וארכיטקטורות מתקדמות כמו VQ-VAE.

הסיבה לעמידותו של האלגוריתם פשוטה: הוא מספק פתרון אלגנטי לבעיה בסיסית – ארגון נתונים לקבוצות. בעידן בו מודלים הופכים מורכבים יותר ויותר, יש ערך רב בכלים פשוטים, יעילים, ומובנים שניתן לשלב בצורה מודולרית בפתרונות גדולים יותר. "הפשטות היא תחכום נעלה" – לאונרדו דה וינצ'י. K-Means הוא דוגמה מושלמת לעיקרון זה בלמידת מכונה.

10 יישומים מעשיים של K-Means: מהתיאוריה לתעשייה

10.1 מהמעבדה אל העולם האמיתי

אלגוריתם K-Means, שנולד בשנות החמישים כרעיון תיאורטי, הפך לאחד הכלים המעשיים והשימושיים ביותר בעולם המודרני. מנועי החיפוש שבהם אנו משתמשים מדי יום, המלצות המוצרים שאנו רואים, ואפילו אבחון רפואי - כולם נשענים על עקרונות הקלסטרינג שלמדנו. בפרק זה נחקור כיצד אלגוריתם מתמטי פשוט הפך לכוח מניע בתעשיות רבות. כפי שציין Hal Varian, הכלכלן הראשי של גוגל: "היכולת לקחת נתונים - להבין אותם, לעבד אותם, להפיק מהם ערך, לדמיין אותם, לתקשר אותם - זו תהיה פיומנות קריטית בעשורים הקרובים" [22]. K-Means הוא אחד הכלים הבסיסיים שמאפשרים לנו לעשות זאת.

10.2 פילוח לקוחות: האם אתה פרסונה או קלסטר?

אחד היישומים המסחריים המרכזיים של K-Means הוא **פילוח לקוחות** (Customer Segmentation) - חלוקת בסיס הלקוחות לקבוצות הומוגניות לצורכי שיווק ממוקד.

הבעיה העסקית:

חברה עם מיליוני לקוחות רוצה להתאים את המסרים השיווקיים שלה לכל לקוח. אבל ליצור מסר ייחודי למיליון לקוחות זה בלתי-אפשרי. הפתרון: חלק את הלקוחות ל- K קבוצות (פלחים), וצור מסר ייחודי לכל פלח.

ייצוג מתמטי:

כל לקוח i מיוצג כווקטור תכונות:

$$(85) \quad \mathbf{x}_i = (\text{age}_i, \text{income}_i, \text{purchases}_i, \text{engagement}_i, \dots)$$

לדוגמה, לקוח יכול להיות:

$$(86) \quad \mathbf{x}_1 = (35, 85000, 12, 0.7, \dots) \in \mathbb{R}^d$$

המשמעות: גיל 35, הכנסה שנתית של 85,000 דולר, 12 רכישות בשנה האחרונה, מעורבות של 70%, וכו'.

יישום K-Means:

- הרץ K-Means עם $K = 5$ (למשל)

- קבל 5 פלחי לקוחות, כל אחד עם מרכז μ_k

- פרש כל פלח: "לקוחות צעירים בעלי הכנסה גבוהה", "משפחות בגיל העמידה", וכו'

- צור אסטרטגיה שיווקית ייעודית לכל פלח

דוגמה מעשית - nozama:

Amazon משתמשת בקלסטרינג לזיהוי דפוסי קנייה. לקוחות שנמצאים באותו קלסטר נוטים לקנות מוצרים דומים, ולכן המלצות המוצרים מבוססות חלקית על "שכנים" בקלסטר [23].

אתגרים מעשיים:

- **נורמליזציה:** תכונות בסקאלות שונות (גיל: 20-80, הכנסה: 20,000-200,000) - חובה לנרמל!

- **בחירת תכונות:** אילו תכונות רלוונטיות? פעילות רשתות חברתיות? היסטוריית תלונות?

- **K דינמי:** מספר הפלחים משתנה עם הזמן וצמיחת החברה

10.3 דחיסת תמונות: פיקסלים כקלסטרים

יישום קלאסי ומרתק של K-Means הוא **דחיסת תמונות** (Image Compression) - הקטנת גודל קובץ תמונה תוך שמירה על איכות סבירה.

הרעיון המרכזי:

תמונה צבעונית היא מטריצה של פיקסלים, כאשר כל פיקסל מיוצג על ידי 3 ערכים: RGB (אדום, ירוק, כחול), כל אחד בטווח $[0, 255]$. תמונה בגודל 1000×1000 מכילה מיליון פיקסלים, כלומר 3 מגה-בתים של מידע.

רוב התמונות אינן משתמשות בכל $16.7 \approx 256^3$ מיליון הצבעים האפשריים - בפועל, יש רק כמה עשרות או מאות צבעים דומיננטיים. K-Means מנצל זאת!

האלגוריתם:

- ייצג כל פיקסל כווקטור $\mathbf{x}_i = (R_i, G_i, B_i) \in \mathbb{R}^3$

- הרץ K-Means עם $K = 16$ (למשל) על כל הפיקסלים

- קבל K צבעים "ייצוגיים" - מרכזי הקלסטרים $\mu_1, \dots, \mu_K$

- החלף כל פיקסל בצבע של הקלסטר שלו

- במקום לאחסן 3 בתים לכל פיקסל, אחסן רק אינדקס קלסטר (4 ביטים עבור $K = 16$)

קצב דחיסה:

ללא דחיסה: 3 בתים $\times n$ פיקסלים $= 3n$ בתים

עם K-Means: $3 \times K$ בתים (לוח צבעים) $+ n \times \log_2(K)$ ביטים (אינדקסים)
עבור $K = 16$ ו- $n = 10^6$:

$$(87) \quad \text{Compression ratio} = \frac{3n}{48 + n \cdot 0.5} \approx 6$$

כלומר, דחיסה פי 6 עם איבוד מינימלי באיכות!

תובנה: זו למעשה **קוונטיזציה וקטורית** (Vector Quantization) - טכניקה מרכזית בדחיסת מידע, ו-K-Means הוא אחד האלגוריתמים הפופולריים ביישומה [24].

10.4 זיהוי חריגות: הנורמלי מול החשוד

זיהוי חריגות (Anomaly Detection) הוא יישום קריטי בתחומים כמו אבטחת מידע, זיהוי הונאות, וניטור מערכות.

הרעיון:

אם הרצנו K-Means והגענו לקלסטרינג "נורמלי", נקודות חדשות שמתנהגות בצורה חריגה יהיו רחוקות ממרכזי הקלסטרים.

האלגוריתם:

- הרץ K-Means על נתוני אימון "נורמליים"

- עבור נקודה חדשה x_{new} , חשב:

$$(88) \quad d_{anomaly} = \min_k \|x_{new} - \mu_k\|$$

- אם $d_{anomaly} > \tau$ (סף מוגדר מראש), סמן כחריגה

דוגמה מעשית - זיהוי הונאות בכרטיסי אשראי:

כל עסקה מיוצגת כווקטור:

$$(89) \quad x = (\text{amount}, \text{time}, \text{location}, \text{merchant_type}, \dots)$$

K-Means מזהה דפוסי קנייה נורמליים. עסקה של 5,000 דולר בשעה 3:00 בבוקר במדינה זרה תהיה רחוקה מכל הקלסטרים - חשד להונאה!

יישום בעולם האמיתי:

חברות כמו PayPal ו-Stripe משתמשות בטכניקות דומות (לרוב משולבות עם למידה מפוקחת) לזיהוי מיליוני הונאות בשנה [25].

10.5 ניתוח מסמכים: מילים כווקטורים

קלסטרינג מסמכים (Document Clustering) הוא יישום חשוב ב-NLP - עיבוד שפה טבעית.

הבעיה:

יש לנו אוסף גדול של מסמכים (מאמרים, חדשות, ציורים), ואנו רוצים לארגן אותם לנושאים.

ייצוג מסמכים - TF-IDF:

כל מסמך מיוצג כווקטור במרחב המילים:

$$(90) \quad x_i = (TF\text{-}IDF_{i,1}, TF\text{-}IDF_{i,2}, \dots, TF\text{-}IDF_{i,V})$$

כאשר V הוא גודל אוצר המילים. TF-IDF (Term Frequency - Inverse Document Fre-)

quency) מודד את חשיבות מילה במסמך:

$$(91) \quad \text{TF-IDF}_{i,j} = \text{TF}_{i,j} \times \log \left(\frac{N}{\text{DF}_j} \right)$$

כאשר:

- $\text{TF}_{i,j}$ - תדירות המילה j במסמך i

- N - מספר כל המסמכים

- DF_j - במכמה מסמכים מופיעה מילה j

יישום K-Means:

- הרץ K-Means על וקטורי TF-IDF

- כל קלסטר מייצג נושא (טכנולוגיה, פוליטיקה, ספורט, וכו')

- מרכז הקלסטר μ_k מכיל את המילים המאפיינות של הנושא

דוגמה - מנועי חיפוש:

גוגל משתמש בקלסטרינג מסמכים לשיפור תוצאות החיפוש - קיבוץ תוצאות דומות, זיהוי נושאים, והמלצות חיפוש [26].

הערה מודרנית:

כיום, במקום TF-IDF, משתמשים בהטמעות (Embeddings) מרשתות נוירונים כמו BERT או GPT - אבל העיקרון נשאר: ווקטורים במרחב, קלסטרינג לפי דמיון.

10.6 ביאוינפורמטיקה: גנים וחלבונים

K-Means משחק תפקיד מרכזי בביאוינפורמטיקה (Bioinformatics) - ניתוח נתונים ביולוגיים.

יישום 1 - ניתוח ביטוי גנים (Gene Expression Analysis):

בניסויי Microarray, מודדים את רמת הביטוי של אלפי גנים בתנאים שונים. כל גן מיוצג כווקטור:

$$(92) \quad \mathbf{x}_i = (\text{expr}_{i,1}, \text{expr}_{i,2}, \dots, \text{expr}_{i,m})$$

כאשר $\text{expr}_{i,j}$ הוא רמת הביטוי של גן i בתנאי j .

K-Means מקבץ גנים עם דפוסי ביטוי דומים - מה שמרמז על תפקוד ביולוגי משותף. זה מאפשר לזהות רשתות גנטיות ומסלולים ביולוגיים [27].

יישום 2 - סיווג סוגי סרטן:

ניתוח ביטוי גנים מאפשר לסווג סרטנים לתת-סוגים מולקולריים. למשל, סרטן השד מתחלק ל-4-5 תת-סוגים עם פרוגנוזות שונות. K-Means על פרופילי ביטוי גנים עוזר לזהות את התת-סוגים ולהתאים טיפול מדויק [28].

תובנה רפואית:

קלסטרינג ביולוגי אינו רק כלי מחקר - הוא משפיע על החלטות קליניות אמיתיות ועל חיי מטופלים.

10.7 המלצות: מה שאחרים אהבו

מערכות המלצה (Recommender Systems) הן אחד היישומים המסחריים הגדולים ביותר של קלסטרינג.

הגישה המבוססת-קלסטרינג:

- ייצג משתמשים כווקטורים של העדפות (דירוגי סרטים, רכישות, האזנות, וכו')

- הרץ K-Means לקיבוץ משתמשים דומים

- המלץ למשתמש על פריטים שאהבו "שכנים" בקלסטר שלו

דוגמה - xilfteN:

Netflix משתמשת בגישות מתוחכמות יותר, אבל הרעיון המרכזי דומה: מצא משתמשים עם טעם דומה, והמלץ על סרטים שהם אהבו. הקלסטרינג עוזר לצמצם את מרחב החיפוש ממיליוני משתמשים לכמה אלפים בקלסטר [29].

yfitopS:

קלסטרינג משתמשים לפי הרגלי האזנה מאפשר ליצור פלייליסטים מותאמים אישית כמו "Discover Weekly" - אחת התכונות הפופולריות ביותר של הפלטפורמה.

10.8 עיבוד תמונה רפואית: איתור גידולים

בהדמיית רפואית (Medical Imaging), K-Means משמש **לפילוח תמונות** (Image Segmentation) - חלוקת תמונה לאזורים בעלי משמעות.

יישום - MRI של המוח:

תמונת MRI מכילה מיליוני פיקסלים. כל פיקסל מיוצג לא רק על ידי עוצמה, אלא גם על ידי מיקום ומרקם:

$$\mathbf{x}_i = (\text{intensity}_i, x_i, y_i, z_i, \text{texture}_i) \quad (93)$$

K-Means עם $K = 4$ מפלח את המוח לרקמות שונות: חומר אפור, חומר לבן, נוזל מוחי-שדרתי, וגידול (אם קיים).

יתרון: אוטומציה של תהליך שבעבר נעשה ידנית על ידי רדיולוגים - חיסכון בזמן ושיפור עקביות [30].

10.9 למידה עמוקה: K-Means כאתחול

באופן מפתיע, K-Means מצא מקום גם בעולם הלמידה העמוקה המודרני.

K-Means לאתחול משקולות רשת:

לפני אימון רשת נוירונים, לעיתים משתמשים ב-K-Means כשלב **אתחול לא-מפוקח** (Un-supervised Pre-training). הרעיון: קבל ייצוג ראשוני של הנתונים בעזרת מרכזי הקלסטרים, ואז השתמש בו לאימון המפוקח [31].

K-Means ככלי להבנה:

בלמידה עמוקה, לעיתים משתמשים ב-K-Means על **שכבות נסתרות** (Hidden Layers) כדי להבין מה הרשת למדה - ויזואליזציה של מרחב הפיצ'רים.

10.10 מגבלות ביישומים מעשיים

למרות הצלחתו, K-Means אינו פתרון קסם לכל בעיה:

- **סקאלביליות:** למרות יעילותו היחסית, K-Means עדיין מתקשה עם מיליארדי נקודות. פתרונות: גרסאות מבוזרות כמו ||K-Means [32]
- **נתונים קטגוריאליים:** K-Means מתוכנן למרחבים מספריים. לקלסטרינג נתונים קטגוריאליים, נדרשים אלגוריתמים כמו K-Modes
- **נתונים זורמים:** בנתונים המשתנים בזמן-אמת, נדרשת גרסה זורמת (Streaming K-Means)
- **פרטיות:** קלסטרינג על נתוני משתמשים עלול לחשוף מידע רגיש - נדרשת **פרטיות דיפרנציאלית** (Differential Privacy)

10.11 סיכום: מדוע K-Means ממשיך לשלוט?

K-Means נשאר פופולרי אחרי כמעט 70 שנה מסיבות פשוטות:

- **פשטות:** קל ליישום, קל להבנה
 - **יעילות:** מהיר גם על מערכי נתונים גדולים
 - **גמישות:** ניתן להתאמה לדומיינים רבים
 - **פרשנות:** התוצאות קלות להבנה ולתקשור
 - **בסיס לאלגוריתמים מתקדמים:** רבים מהשיפורים (כמו ++K-Means, Mini-Batch K-Means) שומרים על הרעיון המרכזי
- כפי שכתב Leo Breiman, אחד מחלוצי הלמידה הסטטיסטית: "שילוב של פשטות ויעילות הופך אלגוריתם לבלתי-ניתן להחלפה" [33]. K-Means הוא ההוכחה החיה לכך. מהלקוחות של אמזון ועד לגידולי המוח, מהמלצות נטפליקס ועד לביטוח הונאות - K-Means ממשיך לעצב את העולם הדיגיטלי שלנו, אלגוריתם אחד בכל פעם.

11 סיכום: K-Means – מסע מהתיאוריה למעשה

11.1 המסע שעברנו

בפרק זה עברנו מסע מרתק מתורת הקלסטרינג הבסיסית ועד ליישומים מתקדמים בלמידת מכונה מודרנית. התחלנו בשאלה פילוסופית פשוטה – כיצד מארגנים נתונים לקבוצות משמעותיות – והגענו לטכנולוגיות חדשניות כמו Vector Quantized Autoencoders ולמידה עצמית-מפוקחת.

הנקודות המרכזיות שסיינו:

1. **בעיית הקלסטרינג (פרק 1.11):** הגדרנו את הבעיה הקומבינטורית של חלוקת נתונים לקבוצות, והבנו מדוע זוהי בעיה NP-hard שדורשת פתרונות אפרוקסימטיביים
2. **גישה פרמטרית (פרק 2.11):** ראינו כיצד K-Means נובע באופן טבעי ממודל Gaussian Mixture תחת הנחות מפשטות
3. **האלגוריתם (פרק 3.11):** הבנו את המבנה האיטרטיבי של K-Means והוכחנו את התכנסותו
4. **יישום מעשי (פרק 4.11):** למדנו על K-Means++, קריטריוני עצירה, וטיפול במקרי קצה
5. **מגבלות (פרק 5.11):** זיהינו מתי K-Means נכשל והבנו את גבולות התחולה שלו
6. **K-Medoids (פרק 6.11):** פגשנו אלגוריתם חלופי העמיד בפני חריגים
7. **ניתוח Silhouette (פרק 7.11):** פיתחנו כלים להערכת איכות הקלסטרינג
8. **הקשר למידת מכונה מודרנית (פרק 8.11):** גילינו כיצד K-Means ממשיך להיות רלוונטי בעידן הרשתות העמוקות
9. **יישומים מעשיים (פרק 9.11):** ראינו מקרי שימוש אמיתיים בתעשייה ובמחקר

11.2 מה למדנו? תובנות מרכזיות

1. **הפשטות היא כוח**
אלגוריתם K-Means, למרות (או דווקא בזכות) פשטותו, הפך לאחד הכלים הנפוצים ביותר בלמידת מכונה. הסיבה פשוטה: הוא עוזר. הוא מהיר, אינטואיטיבי, וניתן ליישום. בעולם בו אלגוריתמים הופכים מורכבים יותר ויותר, K-Means מזכיר לנו שלעיתים הפתרון הפשוט הוא הטוב ביותר.
2. **אין אלגוריתם מושלם**
K-Means איננו "פתרון קסם" לכל בעיית קלסטרינג. הוא מניח קלסטרים כדוריים בעלי צפיפות דומה, רגיש לאתחול, ודורש קביעה מראש של K . ההבנה שכל אלגוריתם מתאים לסוג מסוים של נתונים היא קריטית – לא קיים "אלגוריתם קלסטרינג טוב ביותר" באופן אוניברסלי.

3. מהיסודות התיאורטיים לשימושים מתקדמים

הקשר בין K-Means לבין מודלים הסתברותיים (GMM), רשתות נוירונים (RBF), ולמידה עמוקה (VQ-VAE) מלמד אותנו שאלגוריתמים "פשוטים" עשויים להסתיר מתחתם תיאוריה עמוקה. ההבנה התיאורטית מאפשרת לנו להרחיב ולהכליל את האלגוריתם לתחומים חדשים.

4. חשיבות האתחול

K-Means++ לימד אותנו ששיפור פשוט באתחול יכול לעשות הבדל עצום בביצועים. לעיתים, ההשקעה בשלב ההכנה והאתחול חשובה לא פחות מהאלגוריתם עצמו.

5. הערכה כמותית היא קריטית

מקדם Elbow Method, Silhouette, ומדדים אחרים לימדו אותנו שקלסטרינג אינו אמנות סובייקטיבית. אנו צריכים כלים כמותיים להערכת איכות התוצאות ולהשוואה בין פתרונות שונים.

11.3 טבלת השוואה מקיפה: אלגוריתמי קלסטרינג

להלן השוואה מקיפה בין האלגוריתמים שכיסינו בפרק זה ואלגוריתמים פופולריים אחרים:

סינותנ לדוג	שארמ K	סיגורח	רטסלק תרוצ	תובכרומ	סתירוגלא
לודג	וכ	שיגר	ירודכ	$O(nKdt)$	K-Means
לודג	וכ	שיגר	ירודכ	$O(nKd)$	K-Means++
ינוניב-נטק	וכ	דימע	ירודכ	$O(n^2Kt)$	K-Medoids (PAM)
ינוניב	וכ	שיגר	יטפילא	$O(nK^2dt)$	GMM (EM)
ינוניב	אל	דימע	הרוצ לכ	$O(n \log n)$	DBSCAN
נטק	אל	יולת	הרוצ לכ	$O(n^3)$	Hierarchical
ינוניב-נטק	אל	דימע	ירודכ	$O(n^2)$	Mean Shift
לודג	אל	דימע	הרוצ לכ	$O(n \log n)$	HDBSCAN
ינוניב-נטק	וכ	ינוניב	בכרומ	$O(n^3)$	Spectral

הסבר הקריטריונים:

- מורכבות: n = מספר נקודות, K = מספר קלסטרים, d = מימד, t = איטרציות

- צורת קלסטר: האם האלגוריתם מוגבל לצורות מסוימות

- חריגים: עמידות בפני נקודות חריגות

- K מראש: האם צריך לקבוע את מספר הקלסטרים מראש

- גודל נתונים: התאמה למערכי נתונים שונים

11.4 מתי להשתמש ב-K-Means? מדריך למעשה

□ השתמש ב-K-Means כאשר:

1. הנתונים מתאימים מבחינה גיאומטרית:

- קלסטרים כדוריים או מעט אליפטיים
- גדלים דומים של קלסטרים
- צפיפות אחידה בתוך כל קלסטר

2. יש צורך במהירות:

- מערכי נתונים גדולים ($n > 100,000$)
- צורך בתוצאות מהירות
- אילוצי זמן או זיכרון

3. ידוע מספר הקלסטרים (בקירוב):

- יש ידע תחומי על מבנה הנתונים
- ניתן להעריך K באמצעות Elbow Method או Silhouette

4. הפשטות והפרשנות חשובות:

- צורך בהסבר התוצאות לבעלי עניין
- מרכזי קלסטרים צריכים להיות משמעותיים

5. כשלב ראשון בניתוח:

- חקר ראשוני של מערך הנתונים
- יצירת פיצ'רים ללמידה ממוקדת
- קדם-עיבוד לאלגוריתמים מורכבים יותר

□ אל תשתמש ב-K-Means כאשר:

1. הנתונים לא מתאימים:

- קלסטרים לא-כדוריים (מעגלים, צורות מורכבות)
- הבדלים גדולים בגדלי הקלסטרים
- צפיפות לא-אחידה

2. יש חריגים משמעותיים:

- השתמש ב-K-Medoids או DBSCAN במקום

3. לא ידוע K ואין דרך להעריך אותו:

- השתמש ב-DBSCAN או Hierarchical Clustering

4. הנתונים לא מספריים:

- נתונים קטגוריאליים – השתמש ב-K-Modes

- נתונים מעורבים – השתמש ב-K-Prototypes

5. דרוש קלסטרינג היררכי:

- צורך במבנה עץ של קלסטרים

- השתמש ב-Hierarchical Clustering או HDBSCAN

11.5 תהליך עבודה מומלץ עם K-Means

להלן תהליך עבודה שיטתי לשימוש ב-K-Means בפועל:

11.6 כיוונים עתידיים ומחקר פתוח

אף שאלגוריתם K-Means מבוסס היטב, עדיין קיימים כיוונים מרתקים למחקר ופיתוח:

1. קלסטרינג עמוק (Deep Clustering)

שילוב של למידת ייצוגים עמוקה עם קלסטרינג, כמו ב-DEC (Deep Embedded Clustering) ו-IDEC, מאפשר לימוד סימולטני של הייצוג והקלסטרים. זהו תחום מחקר פעיל עם פוטנציאל רב.

2. קלסטרינג זורם (Streaming Clustering)

עבור נתונים המגיעים בזרם מתמשך, נדרשים אלגוריתמים שיכולים לעדכן קלסטרים באופן דינמי. Online K-Means ו-Incremental K-Means הם צעדים ראשונים, אך יש מקום לשיפור.

3. קלסטרינג עם אילוצים (Constrained Clustering)

במקרים מסוימים, יש אילוצים על הקלסטרינג (למשל, שתי נקודות חייבות להיות באותו קלסטר, או שלא יכולות להיות באותו קלסטר). שילוב אילוצים כאלה ב-K-Means הוא תחום מחקר מעניין.

4. קלסטרינג הוגן (Fair Clustering)

בעידן של בינה מלאכותית אחראית, יש חשיבות רבה להבטיח שקלסטרינג אינו מפלה קבוצות מסוימות. פיתוח גרסאות "הוגנות" של K-Means הוא אתגר חשוב.

5. קלסטרינג בסיסי גרפים (Graph-Based Clustering)

שילוב מידע על קשרים (גרף) עם קלסטרינג מרחבי יכול להוביל לתוצאות טובות יותר בתחומים כמו רשתות חברתיות וביולוגיה חישובית.

11.7 סיום: הלקח המרכזי

אלגוריתם K-Means הוא הרבה יותר מאלגוריתם קלסטרינג פשוט. הוא מייצג פילוסופיה שלמה בלמידת מכונה: פשטות, יעילות, ואלגנטיות מתמטית. מהמקורות שלו בשנות ה-50

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.decomposition import PCA

def kmeans_complete_pipeline(X, k_range=(2, 10),
                             visualize=True):
    """
    K-Means Pipeline
    Parameters:
    X: array-like, shape (n_samples, n_features)
    k_range: tuple
    visualize: bool
    Returns:
    results: dict
    """
    print("===K-Means Complete Pipeline===\n")

    # Step 1: Data Exploration
    print("Step 1: Data Exploration")
    print(f"Dataset shape: {X.shape}")
    print(f"Features: {X.shape[1]}")
    print(f"Samples: {X.shape[0]}")

    # Step 2: Preprocessing
    print("\nStep 2: Preprocessing")
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    print(f"Data normalized (StandardScaler)")

    # Step 3: Selecting K
    print("\nStep 3: Selecting K")
    inertias = []
    silhouettes = []
    db_scores = []

    for k in range(k_range[0], k_range[1]+1):
        kmeans = KMeans(n_clusters=k, init='k-means++',
                        n_init=10, random_state=42)
        labels = kmeans.fit_predict(X_scaled)
```

של המאה ה-20 ועד לשימושים המודרניים בלמידה עמוקה, K-Means הוכיח שאלגוריתמים טובים עומדים במבחן הזמן.

הלקח המרכזי שלנו: אין אלגוריתם "טוב ביותר" באופן אוניברסלי. K-Means מצוין עבור סוגים מסוימים של נתונים ובעיות, אך הוא אינו פתרון קסם. ההצלחה בלמידת מכונה דורשת הבנה עמוקה של הכלים שלנו, ידע מתי להשתמש בהם, ומתי לחפש חלופות. כפי שאמר ג'ורג' בוקס: "כל המודלים שגויים, אך חלקם שימושיים". K-Means הוא דוגמה מושלמת למודל "שגוי" (בהנחותיו המפשטות) אך שימושי ביותר (ביכולתו לפתור בעיות אמיתיות).

במסע שלכם כמדעני נתונים או חוקרי למידת מכונה, זכרו תמיד: הבינו את הכלים שלכם לעומק, דעו את המגבלות שלהם, והיו ביקורתיים בבחירותיכם. זוהי הדרך לבניית מערכות למידת מכונה אמینות, יעילות, ואחראיות.

ולבסוף: קלסטרינג הוא לא רק טכניקה – זוהי דרך לחשוב על העולם. האם הקטגוריות שאנו יוצרים משקפות מציאות אובייקטיבית, או שמא הן משקפות את האופן שבו אנו בוחרים לארגן את המידע? שאלה זו, שהתחלנו בה בפרק 1.11, מלווה אותנו עד הסוף – והיא תמשיך ללוות אותנו בכל פעם שנשתמש באלגוריתמי קלסטרינג.

"הסוף הוא רק התחלה חדשה"

– המסע שלכם עם K-Means רק מתחיל –

12 נספחים: פיתוחים מתמטיים ומעשיים

12.1 נספח א': הוכחה מפורטת של התכנסות K-Means

בפרק 11.3 ראינו הוכחה תמציתית של התכנסות אלגוריתם K-Means. כאן נציג הוכחה מפורטת יותר עם כל השלבים.

משפט (התכנסות): אלגוריתם K-Means מתכנס למינימום לוקלי של פונקציית המטרה J תוך מספר סופי של איטרציות.

הוכחה מפורטת:

חלק א': פונקציית המטרה יורדת מונוטונית

נגדיר את פונקציית המטרה באיטרציה t :

$$(94) \quad J^{(t)} = \sum_{k=1}^K \sum_{i \in C_k^{(t)}} \|\mathbf{x}_i - \boldsymbol{\mu}_k^{(t)}\|^2$$

טענה 1: שלב ההקצאה מקטין או משמר את J .

הוכחה: בשלב ההקצאה באיטרציה t , עבור כל נקודה $\mathbf{x}_i$ אנו בוחרים:

$$(95) \quad c_i^{(t)} = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k^{(t-1)}\|^2$$

כלומר, אנו בוחרים את ההקצאה שממזערת את התרומה של $\mathbf{x}_i$ ל- J . לכן:

$$(96) \quad \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i^{(t)}}^{(t-1)}\|^2 \leq \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i^{(t-1)}}^{(t-1)}\|^2$$

סכימה על כל הנקודות נותנת:

$$(97) \quad J(\mathcal{C}^{(t)}, \boldsymbol{\mu}^{(t-1)}) \leq J(\mathcal{C}^{(t-1)}, \boldsymbol{\mu}^{(t-1)})$$

□ (סיום הוכחת טענה 1)

טענה 2: שלב העדכון מקטין או משמר את J .

הוכחה: בשלב העדכון, עבור קלסטר קבוע $C_k^{(t)}$, אנו מחפשים את $\boldsymbol{\mu}_k$ שממזער:

$$(98) \quad f(\boldsymbol{\mu}_k) = \sum_{i \in C_k^{(t)}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

זוהי פונקציה קמורה ב- $\boldsymbol{\mu}_k$. לכן, המינימום מושג כאשר הגרדיאנט שווה לאפס:

$$(99) \quad \nabla_{\boldsymbol{\mu}_k} f = \nabla_{\boldsymbol{\mu}_k} \sum_{i \in C_k^{(t)}} (\mathbf{x}_i - \boldsymbol{\mu}_k)^T (\mathbf{x}_i - \boldsymbol{\mu}_k) = 0$$

פיתוח:

$$\nabla_{\mu_k} f = \nabla_{\mu_k} \sum_{i \in C_k^{(t)}} (\mathbf{x}_i^T \mathbf{x}_i - 2\mathbf{x}_i^T \mu_k + \mu_k^T \mu_k) \quad (100)$$

$$= \sum_{i \in C_k^{(t)}} (-2\mathbf{x}_i + 2\mu_k) \quad (101)$$

$$= -2 \sum_{i \in C_k^{(t)}} \mathbf{x}_i + 2|C_k^{(t)}| \mu_k \quad (102)$$

השוואה לאפס:

$$|C_k^{(t)}| \mu_k = \sum_{i \in C_k^{(t)}} \mathbf{x}_i \quad (103)$$

לכן:

$$\mu_k^{(t)} = \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} \mathbf{x}_i \quad (104)$$

זהו בדיוק הממוצע החשבוני! מכיוון שזה המינימום של פונקציה קמורה:

$$J(\mathcal{C}^{(t)}, \mu^{(t)}) \leq J(\mathcal{C}^{(t)}, \mu^{(t-1)}) \quad (105)$$

□ (סיום הוכחת טענה 2)

חלק ב': התכנסות תוך מספר סופי של צעדים

מטענות 1 ו-2:

$$J^{(t)} \leq J^{(t-1)} \leq J^{(t-2)} \leq \dots \leq J^{(0)} \quad (106)$$

כלומר, $J^{(t)}$ היא סדרה יורדת מונוטונית. בנוסף:

$$J^{(t)} \geq 0 \quad \text{לכל } t \text{ (סכום של ריבועים)}$$

- יש מספר סופי של הקצאות אפשריות: K^n

לכן, לפי משפט המונוטוניות, הסדרה מתכנסת. יותר מכך, מכיוון שיש רק מספר סופי של הקצאות, האלגוריתם חייב להגיע למצב יציב (בו אין שינוי בהקצאות) תוך מספר סופי של צעדים.

■ (סיום ההוכחה המלאה)

12.2 נספח ב': פיתוח נוסחת ה-Within-Between rettacS

נוכיח את הזהות החשובה: $TSS = WSS + BSS$.

הגדרות:

$$\text{TSS} = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 \quad (\text{Total Sum of Squares}) \quad (107)$$

$$\text{WSS} = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \quad (\text{Within-cluster SS}) \quad (108)$$

$$\text{BSS} = \sum_{k=1}^K |C_k| \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2 \quad (\text{Between-cluster SS}) \quad (109)$$

כאשר $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ הוא הממוצע הכולל.

הוכחה:

נתחיל מ-TSS:

$$\text{TSS} = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 \quad (110)$$

$$= \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 \quad (111)$$

נשתמש בטריק: $\mathbf{x}_i - \boldsymbol{\mu} = (\mathbf{x}_i - \boldsymbol{\mu}_k) + (\boldsymbol{\mu}_k - \boldsymbol{\mu})$

אז:

$$\|\mathbf{x}_i - \boldsymbol{\mu}\|^2 = \|(\mathbf{x}_i - \boldsymbol{\mu}_k) + (\boldsymbol{\mu}_k - \boldsymbol{\mu})\|^2 \quad (112)$$

$$= \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 + 2(\mathbf{x}_i - \boldsymbol{\mu}_k)^T (\boldsymbol{\mu}_k - \boldsymbol{\mu}) + \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2 \quad (113)$$

נסכום על כל הנקודות בקלסטר k :

$$\sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 = \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \quad (114)$$

$$+ 2 \left(\sum_{i \in C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k) \right)^T (\boldsymbol{\mu}_k - \boldsymbol{\mu}) \quad (115)$$

$$+ \sum_{i \in C_k} \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2 \quad (116)$$

שימו לב: $\sum_{i \in C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k) = \sum_{i \in C_k} \mathbf{x}_i - |C_k| \boldsymbol{\mu}_k = |C_k| \boldsymbol{\mu}_k - |C_k| \boldsymbol{\mu}_k = \mathbf{0}$

לכן האיבר האמצעי מתאפס! נותר:

$$(117) \quad \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 = \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 + |C_k| \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2$$

סכימה על כל הקלסטרים:

$$(118) \quad \text{TSS} = \text{WSS} + \text{BSS}$$



משמעות: מזעור WSS (מטרת K-Means) שקול למקסום BSS - ניתן לחשוב על זה כשני צדדים של אותו מטבע.

12.3 נספח ג': מורכבות חישובית מפורטת

נפתח את המורכבות של K-Means בפירוט.

אתחול: $O(K \cdot d)$ - בחירת K מרכזים ב- d מימדים.

איטרציה בודדת:

שלב הקצאה:

- עבור כל נקודה i ועבור כל מרכז k : חישוב $\|x_i - \mu_k\|^2$

- חישוב מרחק אחד: $O(d)$ (סכימת d הפרשים בריבוע)

- סה"כ: $O(n \cdot K \cdot d)$

שלב עדכון:

- עבור כל קלסטר k : סכימת הנקודות וחלוקה ב- $|C_k|$

- סה"כ: $O(n \cdot d)$ (כל נקודה נספרת פעם אחת)

סה"כ לאיטרציה: $O(n \cdot K \cdot d + n \cdot d) = O(n \cdot K \cdot d)$

מספר איטרציות:

במקרה הממוצע: $O(\log n)$ עד $O(\sqrt{n})$ איטרציות.

במקרה הגרוע: עד $O(2^{\sqrt{n}})$ איטרציות (אקספוננציאלי!) [10], אך זה נדיר בפועל.

סיבוכיות כוללת:

ממוצע: $O(n \cdot K \cdot d \cdot T)$ כאשר $T \approx O(\log n)$

לכן: $O(n \cdot K \cdot d \cdot \log n)$

דרישות זיכרון:

- נתונים: $O(n \cdot d)$

- מרכזים: $O(K \cdot d)$

- הקצאות: $O(n)$

סה"כ: $O(n \cdot d + K \cdot d)$

12.4 נספח ד': וריאציות של K-Means

קיימות וריאציות רבות של K-Means המיועדות לבעיות ספציפיות:

1. K-Means++ [9]

שיטת אתחול משופרת שמפזרת את המרכזים ההתחלתיים:

- בחר מרכז ראשון μ_1 באופן אקראי מהנתונים

- עבור $k = 2, \dots, K$:

- חשב עבור כל נקודה $\mathbf{x}_i$ את $D(\mathbf{x}_i) = \min_{j < k} \|\mathbf{x}_i - \mu_j\|^2$

- בחר μ_k עם הסתברות $\propto D(\mathbf{x}_i)$

זה מבטיח ש- $\mathbb{E}[J] \leq O(\log K) \cdot J_{\text{opt}}$ - קירוב לוגריתמי למינימום הגלובלי!

2. Mini-Batch K-Means

לנתונים ענקיים, משתמשים בדגימת מיני-באצ'ים:

- בכל איטרציה, דגום b נקודות (למשל $b = 100$)

- עדכן רק את המרכזים על סמך הדגימה

- מורכבות: $O(b \cdot K \cdot d)$ במקום $O(n \cdot K \cdot d)$

3. K-Medians

במקום ממוצע, שימוש במדיאנה:

$$(119) \quad \mu_k = \text{median}(\{\mathbf{x}_i : i \in C_k\})$$

יתרון: עמידות לחריגים. חיסרון: חישוב מדיאנה יקר יותר.

4. Fuzzy K-Means

במקום הקצאה "קשה" ($c_i \in \{1, \dots, K\}$), הקצאה "רכה":

$$(120) \quad w_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{\|\mathbf{x}_i - \mu_k\|}{\|\mathbf{x}_i - \mu_j\|} \right)^{\frac{2}{m-1}}}$$

כאשר $w_{ik} \in [0, 1]$ ו- $\sum_k w_{ik} = 1$.

5. Spherical K-Means

למסמכים ווקטורים מנורמלים, משתמשים בדמיון קוסינוס במקום מרחק אוקלידי:

$$(121) \quad \text{similarity}(\mathbf{x}_i, \mu_k) = \frac{\mathbf{x}_i^T \mu_k}{\|\mathbf{x}_i\| \|\mu_k\|}$$

12.5 נספח ה': טבלת השוואה - אלגוריתמי קלסטרינג

טבלה 1: השוואה בין אלגוריתמי קלסטרינג מרכזיים

Algorithm	צורת קלסטר	מורכבות	חסרונות	יתרונות
K-Means	קמור, כדורי	$O(nKd)$	דורש K , רגיש לאתחול	מהיר, פשוט, סקאלבילי
K-Medoids	קמור	$O(n^2)$	איטי יותר מ-K-Means	עמיד לחריגים
DBSCAN	שרירותי	$O(n \log n)$	רגיש לפרמטרים	מזהה צורות שרירותיות, עמיד לחריגים
Hierarchical	היררכי	$O(n^2)$	איטי, לא סקאלבילי	לא דורש K , דנדרוגרמה
GMM	אליפטי	$O(nK^2d)$	יקר חישובית	מודל הסתברותי, קלסטרים בגדלים שונים
Spectral	מבוסס-גרף	$O(n^2)$	יקר, בחירת K קשה	צורות קמורות לא-

12.6 נספח ו': נוסחאות מרכזיות - סיכום

פונקציית מטרה:

$$(122) \quad J = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

עדכון מרכזים:

$$(123) \quad \boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

הקצאה:

$$(124) \quad c_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

פיזור בין-קלסטרי:

$$(125) \quad \text{BSS} = \sum_{k=1}^K |C_k| \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2$$

מקדם Silhouette:

$$(126) \quad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

מרחק אוקלידי:

$$(127) \quad d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}$$

מרחק מהטן:

$$(128) \quad d_1(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^d |x_j - y_j|$$

מרחק מינקובסקי:

$$(129) \quad d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p}$$

דמיון קוסינוס:

$$(130) \quad \cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

12.7 נספח ז': משאבים נוספים ללמידה

ספרים מומלצים:

- Christopher Bishop מאת gninrael enihcaM dna noitingoceR nrettaP
- Hastie, Tibshirani, Friedman מאת gninrael lacitsitatS fo stnemele ehT
- gninrael lacitsitatS ot noitcudortnI - גרסה נגישה יותר

קורסים מקוונים:

- Stanford CS229 - Machine Learning
- Coursera - Machine Learning Specialization
- Fast.ai - Practical Deep Learning

ספריות תוכנה:

- (Python) scikit-learn - יישום מצוין של K-Means ווריאציות
- R stats - פונקציית kmeans() מובנית
- MATLAB - פונקציית kmeans()
- Julia MLJ - חבילת למידת מכונה

מאמרים קלאסיים לקריאה נוספת:

- Lloyd (1982) - המאמר המקורי [7]
- MacQueen (1967) - הצגת המונח "K-Means" [34]
- Arthur & Vassilvitskii (2007) - K-Means++ [9]
- Rousseeuw (1987) - מדד Silhouette [16]

כלים לויזואליזציה:

- matplotlib / seaborn - ויזואליזציה ב-Python
- ggplot2 - ויזואליזציה ב-R
- Plotly - ויזואליזציות אינטראקטיביות
- TensorBoard - ניטור ולמידה עמוקה

12.8 נספח ח': סימונים ומוסכמות

סימונים כלליים:

- n - מספר נקודות הנתונים
- d - מימד המרחב (מספר תכונות)
- K - מספר הקלסטרים
- $\mathbf{x}_i \in \mathbb{R}^d$ - נקודת נתונים i
- $\mu_k \in \mathbb{R}^d$ - מרכז הקלסטר k
- C_k - הקלסטר ה- k (קבוצת אינדקסים)
- $|C_k|$ - גודל (מספר נקודות) בקלסטר k
- $c_i \in \{1, \dots, K\}$ - אינדקס הקלסטר של נקודה i

פונקציות ואופרטורים:

- $\|\cdot\|$ - נורמת L2 (אוקלידית)
- $\|\cdot\|_1$ - נורמת L1 (מנהטן)
- $\arg \min$ - האינדקס שממזער
- $\arg \max$ - האינדקס שממקסם
- $\mathbb{E}[\cdot]$ - תוחלת

- $\mathbb{R}^d$ - מרחב אוקלידי d -מימדי

קיצורים נפוצים:

- WSS - Within-cluster Sum of Squares

- BSS - Between-cluster Sum of Squares

- TSS - Total Sum of Squares

- MLE - Maximum Likelihood Estimation

- EM - Expectation-Maximization

- PCA - Principal Component Analysis

12.9 נספח ט': יישום מלא של אלגוריתם PAM

להלן יישום מלא של אלגוריתם PAM (Partitioning Around Medoids) ב-Python, כפי שהוצג בפרק 11.6:

יישום מלא של אלגוריתם MAP

```
\lstinputlisting[language=Python]{main.listing}
```

הסבר מפורט על המימוש:

- **שלב DLIUB (`esahp_dliub_`):** בוחר medoids ראשוניים באופן חכם. תחילה נבחרת הנקודה עם סכום מרחקים מינימלי לכל הנקודות, ואז בכל צעד נוסף נבחרת הנקודה שמביאה לשיפור (רווח) מקסימלי בעלות הכוללת.

- **שלב PAWS (`esahp_paws_`):** בודק את כל ההחלפות האפשריות של medoid קיים בנקודה שאינה medoid. מחשב את השינוי בעלות עבור כל החלפה אפשרית, ומבצע את ההחלפה המשפרת ביותר אם קיימת.

- **חישוב עלות (`tsoc_paws_etupmoc_`):** מחשב את השינוי הכולל בעלות כתוצאה מהחלפת medoid ספציפי. זהו החישוב המרכזי והיקר ביותר באלגוריתם.

- **שיוך לקלסטרים (`slebal_ngissa_`):** משייך כל נקודה ל-medoid הקרוב ביותר אליה על פי מטריצת המרחקים.

- **מורכבות:** המימוש הנוכחי הוא $O(n^2K)$ לאיטרציה, שזה המחיר של האלגוריתם המקורי PAM. גרסאות מודרניות כמו FastPAM משפרות זאת ל- $O(nK)$ [15].

דוגמת שימוש:

הערות יישום:

דוגמת שימוש באלגוריתם MAP

```
import numpy as np
from sklearn.datasets import make_blobs

# סיגירחםעסייטתניסנינותנתריצי
X, _ = make_blobs(n_samples=300, centers=4,
                  cluster_std=1.0, random_state=42)

# סיגירחםעסייטתניסנינותנתריצי
outliers = np.array([[15, 15], [-15, -15], [15, -15]])
X = np.vstack([X, outliers])

# תצרה PAM
pam = PAM(n_clusters=4, max_iter=100)
pam.fit(X)

# תואצותה
print(f"Medoid_indices: {pam.medoid_indices}")
print(f"Cluster_labels: {pam.labels}")
print(f"Medoid_coordinates:")
for idx in pam.medoid_indices:
    print(f" {X[idx]}")
```

1. הקוד תומך במטריצות מרחק מותאמות אישית - ניתן לספק מטריצה משלך
2. ניתן להרחיב בקלות למטריקות מרחק שונות (מנהטן, קוסינוס, וכו')
3. המימוש מתאים למערכי נתונים בגודל בינוני ($n < 5,000$)
4. עבור מערכי נתונים גדולים יותר, מומלץ להשתמש ב-CLARA, CLARANS, או Fast-PAM
5. ההתכנסות מובטחת למינימום לוקלי, אך לא בהכרח לגלובלי

12.10 סיכום הנספחים

נספחים אלה מספקים את הבסיס המתמטי והמעשי המלא להבנת אלגוריתם K-Means, K-Medoids ויישומיהם. מההוכחות המפורטות ועד ליישום מלא של PAM, טבלאות השוואה ומשאבי למידה - כאן תמצאו את כל הכלים הדרושים להמשך מסע הלמידה שלכם בעולם הקלסטרינג ולמידת המכונה.

זכרו: התיאוריה המתמטית היא הבסיס, אך היישום המעשי הוא המטרה. השתמשו בנספחים אלה כנקודת התייחסות בעבודתכם, ותמיד שאפו להבין לא רק "איך" אלא גם "למה" - זוהי הדרך להפוך למדעני נתונים מצוינים.

13 English References

- 1 R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, 179–188, 1936. DOI: [10.1111/j.1469-1809.1936.tb02137.x](https://doi.org/10.1111/j.1469-1809.1936.tb02137.x)
- 2 R. E. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton, NJ: Princeton University Press, 1961.
- 3 R. A. Fisher, *Statistical Methods for Research Workers*. Edinburgh, UK: Oliver and Boyd, 1925.
- 4 R. A. Fisher, "On an absolute criterion for fitting frequency curves," *Messenger of Mathematics*, vol. 41, 155–160, 1912.
- 5 A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, 1–38, 1977.
- 6 P. C. Mahalanobis, "On the generalized distance in statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, 49–55, 1936.
- 7 S. P. Lloyd, "Least squares quantization in pcm," *IEEE Transactions on Information Theory*, vol. 28, no. 2, 129–137, 1982, Originally published as Bell Labs Technical Note in 1957. DOI: [10.1109/TIT.1982.1056489](https://doi.org/10.1109/TIT.1982.1056489)
- 8 E. W. Forgy, "Cluster analysis of multivariate data: Efficiency versus interpretability of classifications," *Biometrics*, vol. 21, 768–769, 1965.
- 9 D. Arthur and S. Vassilvitskii, "K-means++: The advantages of careful seeding," in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '07, Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2007, 1027–1035.
- 10 D. Arthur and S. Vassilvitskii, "How slow is the k-means method?" In *Proceedings of the Twenty-Second Annual Symposium on Computational Geometry*, ACM, 2006, 144–153. DOI: [10.1145/1137856.1137880](https://doi.org/10.1145/1137856.1137880)
- 11 G. E. P. Box, "Science and statistics," *Journal of the American Statistical Association*, vol. 71, no. 356, 791–799, 1976, Contains the famous quote: "All models are wrong, but some are useful". DOI: [10.1080/01621459.1976.10480949](https://doi.org/10.1080/01621459.1976.10480949)
- 12 G. Schwarz, "Estimating the dimension of a model," *The Annals of Statistics*, vol. 6, no. 2, 461–464, 1978, Introduces the Bayesian Information Criterion (BIC). DOI: [10.1214/aos/1176344136](https://doi.org/10.1214/aos/1176344136)

- 13 M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD '96, Introduces the DBSCAN algorithm, AAAI Press, 1996, 226–231.
- 14 L. Kaufman and P. J. Rousseeuw, "Clustering by means of medoids," *Statistical Data Analysis Based on the L1-Norm and Related Methods*, 405–416, 1987.
- 15 E. Schubert and P. J. Rousseeuw, "Faster k-medoids clustering: Improving the pam, clara, and clarans algorithms," *Similarity Search and Applications*, vol. 11807, 171–187, 2019. DOI: [10.1007/978-3-030-32047-8_16](https://doi.org/10.1007/978-3-030-32047-8_16)
- 16 P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, 53–65, 1987. DOI: [10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- 17 D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 1, no. 2, 224–227, 1979. DOI: [10.1109/TPAMI.1979.4766909](https://doi.org/10.1109/TPAMI.1979.4766909)
- 18 T. Calinski and J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics - Theory and Methods*, vol. 3, no. 1, 1–27, 1974. DOI: [10.1080/03610927408827101](https://doi.org/10.1080/03610927408827101)
- 19 A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, 30, Curran Associates, Inc., 2017, 6306–6315.
- 20 M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features," in *European Conference on Computer Vision (ECCV)*, Springer, 2018, 132–149. DOI: [10.1007/978-3-030-01264-9_9](https://doi.org/10.1007/978-3-030-01264-9_9)
- 21 A. Coates and A. Y. Ng, "Learning feature representations with k-means," in *Neural Networks: Tricks of the Trade*, Berlin, Heidelberg: Springer, 2012, 561–580. DOI: [10.1007/978-3-642-35289-8_30](https://doi.org/10.1007/978-3-642-35289-8_30)
- 22 H. R. Varian, "Hal varian on how the web challenges managers," *McKinsey Quarterly*, 2009.
- 23 G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, 76–80, 2003. DOI: [10.1109/MIC.2003.1167344](https://doi.org/10.1109/MIC.2003.1167344)
- 24 R. M. Gray, "Vector quantization," *IEEE ASSP Magazine*, vol. 1, no. 2, 4–29, 1984. DOI: [10.1109/MASSP.1984.1162229](https://doi.org/10.1109/MASSP.1984.1162229)

- 25 S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, 602–613, 2011. DOI: [10.1016/j.dss.2010.08.008](https://doi.org/10.1016/j.dss.2010.08.008)
- 26 D. R. Cutting, D. R. Karger, J. O. Pedersen, and J. W. Tukey, "Scatter/gather: A cluster-based approach to browsing large document collections," in *Proceedings of the 15th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, 1992, 318–329. DOI: [10.1145/133160.133214](https://doi.org/10.1145/133160.133214)
- 27 M. B. Eisen, P. T. Spellman, P. O. Brown, and D. Botstein, "Cluster analysis and display of genome-wide expression patterns," *Proceedings of the National Academy of Sciences*, vol. 95, no. 25, 14863–14868, 1998. DOI: [10.1073/pnas.95.25.14863](https://doi.org/10.1073/pnas.95.25.14863)
- 28 T. Sørli et al., "Gene expression patterns of breast carcinomas distinguish tumor subclasses with clinical implications," *Proceedings of the National Academy of Sciences*, vol. 98, no. 19, 10869–10874, 2001. DOI: [10.1073/pnas.191367098](https://doi.org/10.1073/pnas.191367098)
- 29 J. Bennett and S. Lanning, "The netflix prize," in *Proceedings of KDD Cup and Workshop*, ACM, 2007.
- 30 D. L. Pham, C. Xu, and J. L. Prince, "Current methods in medical image segmentation," *Annual Review of Biomedical Engineering*, vol. 2, 315–337, 2000. DOI: [10.1146/annurev.bioeng.2.1.315](https://doi.org/10.1146/annurev.bioeng.2.1.315)
- 31 A. Coates, H. Lee, and A. Y. Ng, "An analysis of single-layer networks in unsupervised feature learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 15, JMLR Workshop and Conference Proceedings, 2011, 215–223.
- 32 B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," in *Proceedings of the VLDB Endowment*, 5, 2012, 622–633. DOI: [10.14778/2180912.2180915](https://doi.org/10.14778/2180912.2180915)
- 33 L. Breiman, "Statistical modeling: The two cultures," *Statistical Science*, vol. 16, no. 3, 199–231, 2001. DOI: [10.1214/ss/1009213726](https://doi.org/10.1214/ss/1009213726)
- 34 J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, University of California Press, 1967, 281–297.